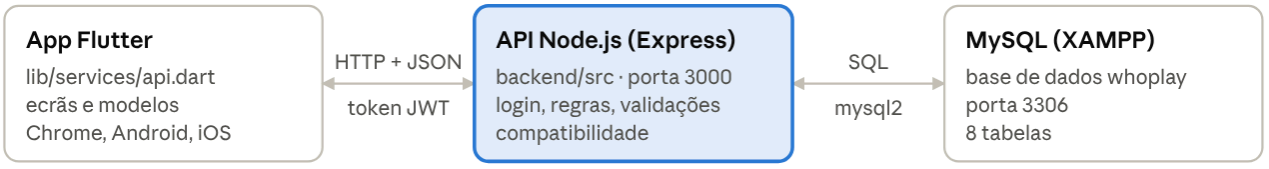
WhoPlay — Guia da base de dados e da API

Oct 28, 2026 · @Sebastião Cardoso

Visão geral

No fim deste guia, a app WhoPlay deixa de usar dados inventados e passa a ler e a gravar tudo numa base de dados MySQL, através de uma API REST em Node.js. É o que pedem os requisitos RNF03, RNF04, RNF05 e RNF10 do relatório do projeto.

A app só fala com a API, e só a API fala com o MySQL



arquitetura · 3 peças

A app nunca fala diretamente com o MySQL: se o fizesse, a palavra-passe da base de dados teria de ir dentro da app, e qualquer pessoa a podia extrair. É a API que verifica quem é o utilizador, o que pode fazer e se os dados estão certos.

O que vão construir

Peça	Tecnologia	Pasta	Ficheiros
Base de dados	MySQL (MariaDB do XAMPP)	`database/`	`schema.sql`, `seed.sql`, `consultas.sql`
API REST	Node.js, Express, mysql2, bcryptjs, JWT	`backend/`	14 ficheiros em `src/`
Ligação na app	Flutter, pacote `http`	`lib/`	4 ficheiros novos e 11 ecrãs atualizados

Requisitos cobertos

Requisito	Onde fica resolvido
RF01 Criar conta / RF02 Iniciar sessão	`POST /auth/registo`, `POST /auth/login`
RF03 Terminar sessão	Botão "Terminar sessão" no Perfil (a app esquece o token)
RF04 Editar perfil	`PUT /perfil`
RF05 / RF06 Modalidades e nível	Tabela `utilizador_modalidades`, `PUT /perfil/modalidades`
RF07 Disponibilidade	Tabela `disponibilidades`, `PUT /perfil/disponibilidade`
RF08 Localização aproximada	Colunas `latitude` e `longitude`, `PUT /perfil`
RF09 Procurar jogadores	`GET /jogadores`
RF10 Compatibilidade	`utils/compatibilidade.js` (5 critérios, fórmula de Haversine)
Objetivo: estatística e método numérico	Média e desvio-padrão em `GET /jogadores`; Haversine para as distâncias
RNF06 Arquitetura modular	Uma rota por ficheiro na API; serviço, modelos e ecrãs separados na app

Pré-requisitos

- **XAMPP** (já instalado em `C:\xampp`). Traz o MariaDB 10.4, que é compatível com MySQL, e o phpMyAdmin.
- **Node.js 18 ou mais recente.** Confirmem com `node --version` (foi testado com o Node 24).
- **A app Flutter do guia anterior**, a funcionar.
- **VS Code** com a extensão **Thunder Client**, para testar a API (o Postman também serve).

Estrutura final do repositório

A app Flutter continua na raiz do repositório. Juntam-se 2 pastas novas: `database/` e `backend/`.

```
WhoPlay-app/
├── database/
│   ├── schema.sql          ← cria as tabelas
│   ├── seed.sql            ← dados de exemplo
│   └── consultas.sql       ← consultas para praticar
├── backend/
│   ├── package.json
│   ├── .env.example        ← modelo das configurações
│   ├── .env                ← configurações reais (não vai para o Git)
│   ├── .gitignore
│   └── src/
│       ├── server.js       ← arranque da API
│       ├── config.js
│       ├── db.js           ← ligação ao MySQL
│       ├── middleware/
│       │   └── auth.js     ← verifica o token
│       ├── utils/
│       │   ├── erros.js
│       │   ├── datas.js
│       │   └── compatibilidade.js
│       └── routes/         ← uma rota por ficheiro
│           ├── auth.js
│           ├── modalidades.js
│           ├── perfil.js
│           ├── jogadores.js
│           ├── jogos.js
│           ├── convites.js
│           └── avisos.js
├── lib/                    ← app Flutter
│   ├── services/api.dart   ← NOVO: todas as chamadas à API
│   ├── data/models.dart    ← NOVO: substitui o mock_data.dart
│   ├── utils/formatos.dart ← NOVO: datas, km e euros em pt-PT
│   ├── widgets/async_view.dart ← NOVO: carregar / erro / conteúdo
│   └── screens/            ← ecrãs atualizados + register_screen.dart
└── pubspec.yaml            ← + pacote http
```

Ordem de trabalho

1. **Partes 1 a 5 (base de dados):** arrancar o MySQL, perceber o modelo, criar as tabelas, carregar os dados e praticar SQL. Isto pode ser feito por toda a equipa ao mesmo tempo.
2. **Partes 6 a 11 (API):** criar o servidor, as rotas e testá-las.
3. **Parte 12 (app):** ligar os ecrãs à API.

Parte 1 — Arrancar o MySQL no XAMPP

O MySQL tem de estar a correr sempre que usam a API ou a app. No XAMPP, basta 1 clique.

1. Abram o **XAMPP Control Panel** (`C:\xampp\xampp-control.exe`).
2. Na linha **MySQL**, carreguem em **Start**. O nome fica verde e aparece a porta **3306**.
3. Na linha **Apache**, carreguem também em **Start**. Só é preciso para o phpMyAdmin; a API não usa o Apache.
4. Abram o phpMyAdmin no browser: http://localhost/phpmyadmin.

No XAMPP, o utilizador do MySQL é `root` e não tem palavra-passe. Para um projeto escolar, no vosso PC, isto chega; num servidor real, nunca.

Usar o MySQL pela linha de comandos (opcional)

O XAMPP traz o cliente `mysql` . No Git Bash, confirmem que o servidor responde:

```
/c/xampp/mysql/bin/mysql.exe -u root -e "SELECT VERSION();"
```

Deve aparecer algo como `10.4.32-MariaDB` . O XAMPP usa o **MariaDB**, um "irmão" do MySQL com os mesmos comandos SQL. Tudo neste guia funciona igual no MySQL 8.

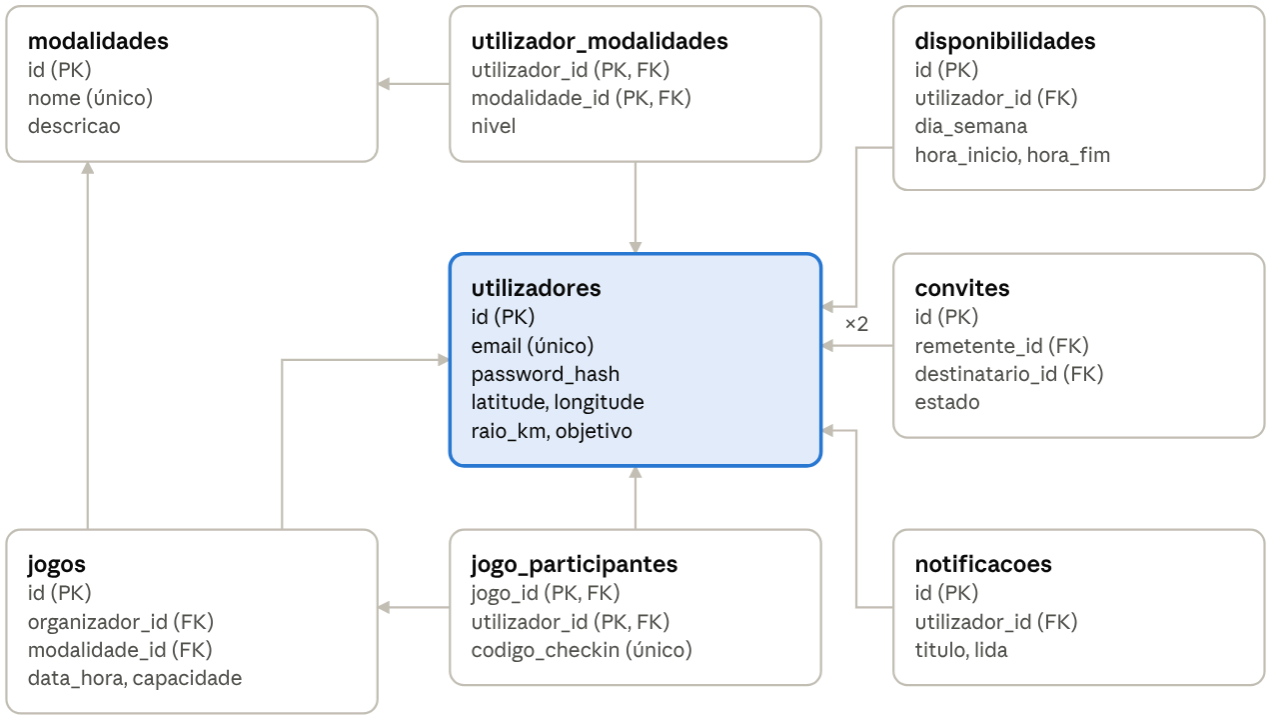
Se o MySQL não arrancar

Sintoma no XAMPP	Causa	Solução
"Port 3306 in use"	Outro MySQL (Laragon, MySQL Server, Workbench) já está a usar a porta	Parem o outro programa, ou mudem a porta em **Config → my.ini** (`port=3307`) e no `.env` da API (`DB_PORT=3307`)
Arranca e pára logo a seguir	Ficheiros da base de dados danificados, normalmente depois de desligar o PC sem parar o XAMPP	Vejam **Logs**; muitas vezes resolve-se apagando `C:\xampp\mysql\data\aria_log_control`
O phpMyAdmin não abre	O Apache está parado ou a porta 80 está ocupada	Arranquem o Apache; se a porta 80 estiver ocupada (por exemplo, pelo Skype), mudem-na em **Config → httpd.conf**

Parte 2 — Modelo de dados

A base de dados `whoplay` tem 8 tabelas. A tabela central é `utilizadores`, e todas as outras apontam para ela. Cada tabela saiu de um requisito ou de um ecrã do mockup.

Todas as tabelas ligam-se a utilizadores



Cada seta é uma chave estrangeira (FK) e aponta para a tabela a que se refere.

modelo entidade-relação · 8 tabelas

Os `convites` têm 2 chaves para `utilizadores` (quem envia e quem recebe) e também uma para `modalidades`, que não está desenhada para o diagrama ficar mais legível.

As 8 tabelas

Tabela	Uma linha é…	Colunas importantes	Ecrãs que a usam
`utilizadores`	uma conta	`email` (único), `password_hash`, `latitude`, `longitude`, `raio_km`, `objetivo`	Login, Perfil
`modalidades`	um desporto	`nome` (único), `descricao` ("2 ou 4 jogadores")	Onboarding, Criar jogo

Tabela	Uma linha é…	Colunas importantes	Ecrãs que a usam
`utilizador_modalidades`	"esta pessoa joga este desporto, com este nível"	`nivel` (Iniciante, Intermédio, Avançado)	Perfil, Jogadores
`disponibilidades`	um horário livre semanal	`dia_semana` (1 = segunda … 7 = domingo), `hora_inicio`, `hora_fim`	Perfil, compatibilidade
`jogos`	um jogo marcado	`data_hora`, `local_nome`, `capacidade`, `jogadores_externos`, `preco`, `estado`	Criar jogo, Mapa, Detalhe
`jogo_participantes`	"esta pessoa vai a este jogo"	`codigo_checkin` (o texto do QR), `checkin_em`	Detalhe, Check-in
`convites`	um convite de uma pessoa para outra	`remetente_id`, `destinatario_id`, `estado` (pendente, aceite, recusado)	Convites
`notificacoes`	um aviso	`titulo`, `detalhe`, `lida`	Convites (avisos)

Conceitos que vão encontrar no `schema.sql`

- **Chave primária (`PRIMARY KEY`)**: identifica cada linha. Nas tabelas normais é um `id` com `AUTO_INCREMENT` (o MySQL dá 1, 2, 3… sozinho).
- **Chave estrangeira (`FOREIGN KEY`)**: liga uma coluna à chave de outra tabela. Por exemplo, `jogos.organizador_id` → `utilizadores.id`. O MySQL recusa um jogo cujo organizador não exista.
- **`ON DELETE CASCADE`**: se apagarem um utilizador, as linhas dele noutras tabelas (desportos, horários, convites…) são apagadas com ele.
- **Tabela de ligação N:N**: um utilizador joga vários desportos, e um desporto tem vários utilizadores. A tabela `utilizador_modalidades` guarda cada par, e a chave primária é o par (`utilizador_id`, `modalidade_id`), o que impede repetir o mesmo desporto. O mesmo acontece em `jogo_participantes`.

- **`ENUM`**: a coluna só aceita os valores da lista. Por exemplo, `nivel ENUM('Iniciante','Intermédio','Avançado')`.
- **`CHECK`**: uma regra que o MySQL verifica em cada gravação. Por exemplo, `hora_fim > hora_inicio` ou `remetente_id <> destinatario_id` (ninguém se convida a si próprio).
- **`UNIQUE`**: não pode haver 2 linhas com o mesmo valor. É o caso do email e do código do QR.
- **`utf8mb4`**: o conjunto de caracteres que guarda bem os acentos ("Ténis", "Sebastião") e até emojis.

Decisões de desenho

- **A palavra-passe nunca é guardada.** Guarda-se o `password_hash`, um resultado do bcrypt que não se consegue reverter. No login, a API calcula o hash do que foi escrito e compara os 2.
- **As vagas não são guardadas, são calculadas:** `capacidade − jogadores_externos − número de participantes`. Assim nunca ficam desatualizadas. `jogadores_externos` são as pessoas que vão ao jogo mas não usam a app (no Criar jogo, "1 vaga de 4" quer dizer que o organizador já tem 2 amigos de fora).
- **O check-in fica em `jogo_participantes`:** cada participante recebe um código único (UUID) quando entra no jogo, e esse código é o conteúdo do QR Code.
- **A compatibilidade não é guardada:** é calculada na API sempre que se procura, porque muda quando alguém altera o horário ou a localização.

Parte 3 — Criar a base de dados (`schema.sql`)

Criem a pasta `database/` na raiz do repositório e, dentro dela, o ficheiro `schema.sql` com o conteúdo abaixo. O script apaga a base de dados `whoplay` (se existir) e volta a criá-la vazia, por isso podem corrê-lo as vezes que quiserem para recomeçar do zero.

Como correr o script

No phpMyAdmin: separador **Importar** → **Escolher ficheiro** → `schema.sql` → **Importar**. Do lado esquerdo aparece a base de dados `whoplay`, com 8 tabelas.

Na linha de comandos (Git Bash, na raiz do repositório):

```
/c/xampp/mysql/bin/mysql.exe -u root --default-character-set=utf8mb4 < database/schema.sql
```

O `--default-character-set=utf8mb4` garante que os acentos dos `ENUM` ("Intermédio", "Amigável") são lidos corretamente.

`database/schema.sql`

```
-- ============================================================
-- WhoPlay — estrutura da base de dados (MySQL 8 / MariaDB 10.4+)
-- Correr no phpMyAdmin (separador SQL) ou:
--   mysql -u root < database/schema.sql
-- ATENÇÃO: apaga a base de dados whoplay se já existir.
-- ============================================================

DROP DATABASE IF EXISTS whoplay;
CREATE DATABASE whoplay CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE whoplay;

-- ------------------------------------------------------------
-- 1. Utilizadores (RF01–RF04, RF08)
-- ------------------------------------------------------------
CREATE TABLE utilizadores (
  id             INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  nome           VARCHAR(80)   NOT NULL,
  email          VARCHAR(120)  NOT NULL,
  password_hash  VARCHAR(255)  NOT NULL,
  localidade     VARCHAR(80)   NULL,
  latitude       DECIMAL(9,6)  NULL,
  longitude      DECIMAL(9,6)  NULL,
  raio_km        TINYINT UNSIGNED NOT NULL DEFAULT 5,
  objetivo       ENUM('Jogos amigáveis','Jogos competitivos','Treino') NOT
NULL DEFAULT 'Jogos amigáveis',
  avaliacao      DECIMAL(2,1)  NULL,
  criado_em      DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_utilizadores_email (email)
) ENGINE=InnoDB;

-- ------------------------------------------------------------
-- 2. Modalidades (lista de desportos da app)
-- ------------------------------------------------------------
CREATE TABLE modalidades (
  id         TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome       VARCHAR(40)  NOT NULL,
  descricao  VARCHAR(60)  NOT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_modalidades_nome (nome)
) ENGINE=InnoDB;
```

```
-- ------------------------------------------------------------
-- 3. Que modalidades cada utilizador joga e com que nível (RF05, RF06)
--    Tabela de ligação N:N entre utilizadores e modalidades.
-- ------------------------------------------------------------
CREATE TABLE utilizador_modalidades (
  utilizador_id  INT UNSIGNED     NOT NULL,
  modalidade_id  TINYINT UNSIGNED NOT NULL,
  nivel          ENUM('Iniciante','Intermédio','Avançado') NOT NULL DEFAULT
'Iniciante',
  PRIMARY KEY (utilizador_id, modalidade_id),
  CONSTRAINT fk_um_utilizador FOREIGN KEY (utilizador_id) REFERENCES
utilizadores (id) ON DELETE CASCADE,
  CONSTRAINT fk_um_modalidade FOREIGN KEY (modalidade_id) REFERENCES
modalidades (id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------
-- 4. Disponibilidade semanal (RF07). dia_semana: 1 = segunda … 7 = domingo
-- ------------------------------------------------------------
CREATE TABLE disponibilidades (
  id             INT UNSIGNED     NOT NULL AUTO_INCREMENT,
  utilizador_id  INT UNSIGNED     NOT NULL,
  dia_semana     TINYINT UNSIGNED NOT NULL,
  hora_inicio    TIME             NOT NULL,
  hora_fim       TIME             NOT NULL,
  PRIMARY KEY (id),
  KEY idx_disp_utilizador (utilizador_id, dia_semana),
  CONSTRAINT fk_disp_utilizador FOREIGN KEY (utilizador_id) REFERENCES
utilizadores (id) ON DELETE CASCADE,
  CONSTRAINT ck_disp_dia CHECK (dia_semana BETWEEN 1 AND 7),
  CONSTRAINT ck_disp_horas CHECK (hora_fim > hora_inicio)
) ENGINE=InnoDB;

-- ------------------------------------------------------------
-- 5. Jogos criados pelos utilizadores
--    jogadores_externos = pessoas que já vão mas não usam a app.
--    Vagas livres = capacidade - jogadores_externos - participantes.
-- ------------------------------------------------------------
CREATE TABLE jogos (
  id                  INT UNSIGNED     NOT NULL AUTO_INCREMENT,
  organizador_id      INT UNSIGNED     NOT NULL,
  modalidade_id       TINYINT UNSIGNED NOT NULL,
  data_hora           DATETIME         NOT NULL,
  local_nome          VARCHAR(100)     NOT NULL,
  latitude            DECIMAL(9,6)     NULL,
```

```
  longitude           DECIMAL(9,6)     NULL,
  nivel               ENUM('Iniciante','Intermédio','Avançado') NOT NULL,
  formato             ENUM('Amigável','Competitivo') NOT NULL DEFAULT
'Amigável',
  capacidade          TINYINT UNSIGNED NOT NULL,
  jogadores_externos  TINYINT UNSIGNED NOT NULL DEFAULT 0,
  preco               DECIMAL(6,2)     NOT NULL DEFAULT 0,
  estado              ENUM('aberto','cancelado','terminado') NOT NULL DEFAULT
'aberto',
  criado_em           DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  KEY idx_jogos_data (data_hora),
  CONSTRAINT fk_jogos_organizador FOREIGN KEY (organizador_id) REFERENCES
utilizadores (id) ON DELETE CASCADE,
  CONSTRAINT fk_jogos_modalidade FOREIGN KEY (modalidade_id) REFERENCES
modalidades (id),
  CONSTRAINT ck_jogos_capacidade CHECK (capacidade BETWEEN 2 AND 30)
) ENGINE=InnoDB;

-- ------------------------------------------------------------
-- 6. Quem vai a cada jogo + check-in por QR Code
-- ------------------------------------------------------------
CREATE TABLE jogo_participantes (
  jogo_id         INT UNSIGNED NOT NULL,
  utilizador_id   INT UNSIGNED NOT NULL,
  entrou_em       DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  codigo_checkin  CHAR(36)     NOT NULL,
  checkin_em      DATETIME     NULL,
  PRIMARY KEY (jogo_id, utilizador_id),
  UNIQUE KEY uq_part_codigo (codigo_checkin),
  CONSTRAINT fk_part_jogo FOREIGN KEY (jogo_id) REFERENCES jogos (id) ON
DELETE CASCADE,
  CONSTRAINT fk_part_utilizador FOREIGN KEY (utilizador_id) REFERENCES
utilizadores (id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------
-- 7. Convites entre jogadores
-- ------------------------------------------------------------
CREATE TABLE convites (
  id               INT UNSIGNED     NOT NULL AUTO_INCREMENT,
  remetente_id     INT UNSIGNED     NOT NULL,
  destinatario_id  INT UNSIGNED     NOT NULL,
  modalidade_id    TINYINT UNSIGNED NOT NULL,
  data_hora        DATETIME         NOT NULL,
  estado           ENUM('pendente','aceite','recusado') NOT NULL DEFAULT
```

```
'pendente',
  criado_em        DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
  respondido_em    DATETIME         NULL,
  PRIMARY KEY (id),
  KEY idx_conv_destinatario (destinatario_id, estado),
  CONSTRAINT fk_conv_remetente FOREIGN KEY (remetente_id) REFERENCES
utilizadores (id) ON DELETE CASCADE,
  CONSTRAINT fk_conv_destinatario FOREIGN KEY (destinatario_id) REFERENCES
utilizadores (id) ON DELETE CASCADE,
  CONSTRAINT fk_conv_modalidade FOREIGN KEY (modalidade_id) REFERENCES
modalidades (id),
  CONSTRAINT ck_conv_pessoas CHECK (remetente_id <> destinatario_id)
) ENGINE=InnoDB;

-- ------------------------------------------------------------
-- 8. Avisos (notificações dentro da app)
-- ------------------------------------------------------------
CREATE TABLE notificacoes (
  id             INT UNSIGNED NOT NULL AUTO_INCREMENT,
  utilizador_id  INT UNSIGNED NOT NULL,
  titulo         VARCHAR(150) NOT NULL,
  detalhe        VARCHAR(150) NOT NULL DEFAULT '',
  lida           BOOLEAN      NOT NULL DEFAULT FALSE,
  criado_em      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  KEY idx_notif_utilizador (utilizador_id, lida),
  CONSTRAINT fk_notif_utilizador FOREIGN KEY (utilizador_id) REFERENCES
utilizadores (id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Confirmar

No phpMyAdmin, abram a base de dados `whoplay` → **Estrutura**: têm de aparecer as 8 tabelas, todas vazias. No separador **Designer** podem ver as ligações entre tabelas desenhadas pelo próprio phpMyAdmin.

Parte 4 — Dados de exemplo (`seed.sql`)

O `seed.sql` enche a base de dados com as pessoas e os jogos do mockup, para terem com que testar desde o primeiro minuto. Corram-no sempre **depois** do `schema.sql`.

```
/c/xampp/mysql/bin/mysql.exe -u root --default-character-set=utf8mb4 <
database/seed.sql
```

No phpMyAdmin: **Importar** → `seed.sql`.

O que fica na base de dados

Quem	Email	Onde	Joga	Distância ao Sebastião
Sebastião	sebastiao@email.com	Moscavide	Padel (Intermédio), Ténis (Iniciante)	—
Tchali	tchali@email.com	Olivais	Padel (Intermédio)	1,8 km
Helio	helio@email.com	Alvalade	Padel e Ténis (Intermédio)	3,1 km
Cristiano	cristiano@email.com	Marvila	Padel (Avançado), Futebol (Intermédio)	4,2 km

A palavra-passe de todos é **12345678**. Fica também 1 jogo de Padel no Padel Club X no próximo sábado às 10:00 (Sebastião, Helio e Cristiano, com 1 vaga livre), 3 convites e 3 avisos do Sebastião.

Truques usados no script:

- `SET @pass = '...'` cria uma variável com o hash bcrypt de "12345678", para não o repetir 4 vezes. Esse hash foi gerado pela própria API (Parte 7).
- `(5 - WEEKDAY(CURDATE()) + 7) % 7` calcula quantos dias faltam até sábado (no MySQL, `WEEKDAY` dá 0 para segunda e 5 para sábado). Assim, o jogo de exemplo fica sempre no futuro, seja qual for o dia em que correm o script.
- `UUID()` gera o código único do QR de cada participante.
- `NOW() - INTERVAL 5 MINUTE` põe o aviso "há 5 minutos".

`database/seed.sql`

```
-- ============================================================
-- WhoPlay — dados de exemplo (os mesmos do mockup)
-- Correr DEPOIS do schema.sql:
--   mysql -u root < database/seed.sql
-- Palavra-passe de todos os utilizadores: 12345678
-- ============================================================

USE whoplay;

-- Modalidades ------------------------------------------------
INSERT INTO modalidades (id, nome, descricao) VALUES
  (1, 'Padel',       '2 ou 4 jogadores'),
  (2, 'Ténis',       '2 ou 4 jogadores'),
```

```
  (3, 'Futebol',     '5, 7 ou 11 por equipa'),
  (4, 'BJJ',         'Parceiro de treino'),
  (5, 'Basquetebol', '3x3 ou 5x5'),
  (6, 'Voleibol',    'Praia ou pavilhão'),
  (7, 'Running',     'Companhia de corrida'),
  (8, 'Ginásio',     'Parceiro de treino');

-- Utilizadores ------------------------------------------------
-- O hash abaixo é o bcrypt de "12345678" (gerado pela API).
SET @pass = '$2b$10$pmW7t7Clw0vlmN2AmEkgd.CI3VDEi4DeFultw7XPSs8aZHl5Szzw2';

INSERT INTO utilizadores (id, nome, email, password_hash, localidade,
latitude, longitude, raio_km, objetivo, avaliacao) VALUES
  (1, 'Sebastião', 'sebastiao@email.com', @pass, 'Moscavide', 38.779000,
-9.103000, 5, 'Jogos amigáveis',    4.8),
  (2, 'Tchali',    'tchali@email.com',    @pass, 'Olivais',   38.795200,
-9.103000, 5, 'Jogos amigáveis',    4.7),
  (3, 'Helio',     'helio@email.com',     @pass, 'Alvalade',  38.779000,
-9.138700, 5, 'Jogos amigáveis',    4.5),
  (4, 'Cristiano', 'cristiano@email.com', @pass, 'Marvila',   38.741200,
-9.103000, 5, 'Jogos competitivos', 4.2);

-- Desportos e nível de cada um -------------------------------
INSERT INTO utilizador_modalidades (utilizador_id, modalidade_id, nivel)
VALUES
  (1, 1, 'Intermédio'), (1, 2, 'Iniciante'),
  (2, 1, 'Intermédio'),
  (3, 1, 'Intermédio'), (3, 2, 'Intermédio'),
  (4, 1, 'Avançado'),   (4, 3, 'Intermédio');

-- Disponibilidade (1 = segunda … 7 = domingo) ------------------
INSERT INTO disponibilidades (utilizador_id, dia_semana, hora_inicio,
hora_fim) VALUES
  -- Sebastião
  (1, 1, '18:00', '21:00'), (1, 3, '18:00', '22:00'), (1, 6, '10:00',
'15:00'),
  -- Tchali
  (2, 1, '18:00', '21:00'), (2, 2, '19:00', '21:00'), (2, 3, '18:00',
'21:00'), (2, 4, '19:00', '21:00'),
  (2, 5, '19:00', '21:00'), (2, 6, '10:00', '13:30'), (2, 7, '19:00',
'21:00'),
  -- Helio
  (3, 1, '18:00', '20:00'), (3, 2, '18:00', '20:00'), (3, 3, '18:00',
'20:00'), (3, 4, '18:00', '20:00'),
  (3, 5, '18:00', '20:00'), (3, 6, '10:00', '12:00'), (3, 7, '18:00',
'20:00'),
```

```
  -- Cristiano
  (4, 1, '20:00', '22:00'), (4, 2, '20:00', '22:00'), (4, 3, '20:00',
'22:00'), (4, 4, '20:00', '22:00'),
  (4, 5, '20:00', '22:00'), (4, 6, '14:00', '16:00'), (4, 7, '20:00',
'22:00');

-- Jogo de Padel no próximo sábado às 10:00 ---------------------
-- (5 - WEEKDAY(hoje) + 7) % 7 = dias que faltam até sábado
INSERT INTO jogos (id, organizador_id, modalidade_id, data_hora, local_nome,
latitude, longitude,
                   nivel, formato, capacidade, jogadores_externos, preco)
VALUES
  (1, 1, 1,
   TIMESTAMP(CURDATE() + INTERVAL ((5 - WEEKDAY(CURDATE()) + 7) % 7) DAY,
'10:00:00'),
   'Padel Club X', 38.757400, -9.103000, 'Intermédio', 'Amigável', 4, 0,
8.00);

INSERT INTO jogo_participantes (jogo_id, utilizador_id, codigo_checkin)
VALUES
  (1, 1, UUID()), (1, 3, UUID()), (1, 4, UUID());

-- Convites -----------------------------------------------------
INSERT INTO convites (remetente_id, destinatario_id, modalidade_id,
data_hora, estado, respondido_em) VALUES
  -- Sebastião convidou o Tchali para hoje às 19:00 (à espera)
  (1, 2, 1, TIMESTAMP(CURDATE(), '19:00:00'), 'pendente', NULL),
  -- Cristiano convidou o Sebastião para quarta às 18:00 (por responder)
  (4, 1, 1, TIMESTAMP(CURDATE() + INTERVAL ((2 - WEEKDAY(CURDATE()) + 7) % 7)
DAY, '18:00:00'), 'pendente', NULL),
  -- Sebastião convidou o Helio e ele aceitou
  (1, 3, 1, TIMESTAMP(CURDATE(), '18:00:00'), 'aceite', NOW() - INTERVAL 5
MINUTE);

-- Avisos do Sebastião ------------------------------------------
INSERT INTO notificacoes (utilizador_id, titulo, detalhe, lida, criado_em)
VALUES
  (1, 'O Helio aceitou o teu convite', '', FALSE, NOW() - INTERVAL 5 MINUTE),
  (1, 'Falta 1 jogador para o teu jogo', 'Padel · sábado às 10:00', FALSE,
NOW() - INTERVAL 1 HOUR),
  (1, 'Encontrámos 3 jogadores compatíveis perto de ti', '', TRUE, NOW() -
INTERVAL 1 DAY);
```

Recomeçar do zero

Sempre que quiserem a base de dados de volta ao estado inicial (depois de muitos testes, por exemplo), corram os 2 scripts seguidos:

```
/c/xampp/mysql/bin/mysql.exe -u root --default-character-set=utf8mb4 < database/schema.sql
/c/xampp/mysql/bin/mysql.exe -u root --default-character-set=utf8mb4 < database/seed.sql
```

Parte 5 — Praticar SQL

Antes de escreverem a API, pratiquem as consultas que ela vai fazer. Guardem o ficheiro abaixo como `database/consultas.sql` e corram as consultas **uma de cada vez**: no phpMyAdmin, abram a base de dados `whoplay` → separador **SQL**, colem uma consulta e carreguem em **Executar**.

Nº	O que pratica	Resultado esperado com o `seed.sql`
1	`SELECT` e `ORDER BY`	8 modalidades por ordem alfabética
2	`JOIN` de 3 tabelas	Sebastião: Padel (Intermédio) e Ténis (Iniciante)
3	Funções (`ELT`, `TIME_FORMAT`)	Segunda 18h–21h, Quarta 18h–22h, Sábado 10h–15h
4	Cálculo numérico (Haversine) e `HAVING`	Tchali 1,8 km, Helio 3,1 km, Cristiano 4,2 km
5	Subconsulta	Jogo 1, Padel Club X, 1 vaga
6	`WHERE` com 2 condições	Convite do Cristiano
7	`COUNT`	2 avisos por ler
8	Estatística: `GROUP BY`, `AVG`, `SUM`	Padel: 4 jogadores, média de avaliação 4,55, 1 avançado
9	`UPDATE`	1 linha alterada
10	`INSERT`	1 linha inserida
11	`DELETE` (desfaz os passos 9 e 10)	1 linha apagada e 1 alterada

A fórmula de Haversine (consulta 4) calcula a distância entre 2 pontos na superfície da Terra a partir das latitudes e longitudes. É o método numérico pedido nos objetivos do projeto:

```
d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\varphi}{2}\right) +
\cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}
{2}\right)}\right)
```

Onde R = 6371 km (raio da Terra), φ é a latitude e λ a longitude, ambas em radianos. A API faz exatamente a mesma conta em JavaScript (Parte 9).

`database/consultas.sql`

```
-- ============================================================
-- WhoPlay — consultas de prática (as mesmas que a API usa)
-- Corram uma de cada vez no phpMyAdmin (separador SQL).
-- ============================================================

USE whoplay;

-- 1. Todas as modalidades, por ordem alfabética
SELECT id, nome, descricao
FROM modalidades
ORDER BY nome;

-- 2. Perfil do Sebastião com os desportos e níveis (JOIN)
SELECT u.nome, u.localidade, m.nome AS modalidade, um.nivel
FROM utilizadores u
JOIN utilizador_modalidades um ON um.utilizador_id = u.id
JOIN modalidades m             ON m.id = um.modalidade_id
WHERE u.email = 'sebastiao@email.com';

-- 3. Disponibilidade do Sebastião, com o nome do dia
SELECT ELT(dia_semana,
'Segunda','Terça','Quarta','Quinta','Sexta','Sábado','Domingo') AS dia,
       TIME_FORMAT(hora_inicio, '%Hh') AS das,
       TIME_FORMAT(hora_fim, '%Hh')    AS ate
FROM disponibilidades
WHERE utilizador_id = 1
ORDER BY dia_semana;

-- 4. Jogadores de Padel a menos de 5 km do Sebastião (fórmula de Haversine)
--    6371 = raio da Terra em km
SELECT u.id, u.nome, um.nivel,
       ROUND(6371 * 2 * ASIN(SQRT(
```

```
          POWER(SIN(RADIANS(u.latitude - eu.latitude) / 2), 2) +
          COS(RADIANS(eu.latitude)) * COS(RADIANS(u.latitude)) *
          POWER(SIN(RADIANS(u.longitude - eu.longitude) / 2), 2)
        )), 1) AS distancia_km
FROM utilizadores u
JOIN utilizador_modalidades um ON um.utilizador_id = u.id AND
um.modalidade_id = 1
JOIN utilizadores eu           ON eu.id = 1
WHERE u.id <> eu.id
HAVING distancia_km <= 5
ORDER BY distancia_km;

-- 5. Jogos abertos com o número de vagas livres (subconsulta)
SELECT j.id, m.nome AS modalidade, j.local_nome,
       DATE_FORMAT(j.data_hora, '%d/%m %H:%i') AS quando,
       j.capacidade - j.jogadores_externos
         - (SELECT COUNT(*) FROM jogo_participantes p WHERE p.jogo_id = j.id)
AS vagas
FROM jogos j
JOIN modalidades m ON m.id = j.modalidade_id
WHERE j.estado = 'aberto' AND j.data_hora > NOW()
ORDER BY j.data_hora;

-- 6. Convites recebidos pelo Sebastião que ainda estão por responder
SELECT c.id, r.nome AS de, m.nome AS modalidade, c.data_hora
FROM convites c
JOIN utilizadores r ON r.id = c.remetente_id
JOIN modalidades m  ON m.id = c.modalidade_id
WHERE c.destinatario_id = 1 AND c.estado = 'pendente';

-- 7. Quantos avisos por ler tem o Sebastião (COUNT)
SELECT COUNT(*) AS por_ler
FROM notificacoes
WHERE utilizador_id = 1 AND lida = FALSE;

-- 8. Estatística: jogadores e avaliação média por modalidade (GROUP BY)
SELECT m.nome AS modalidade,
       COUNT(*)                   AS jogadores,
       ROUND(AVG(u.avaliacao), 2) AS avaliacao_media,
       SUM(um.nivel = 'Avançado') AS avancados
FROM utilizador_modalidades um
JOIN modalidades m  ON m.id = um.modalidade_id
JOIN utilizadores u ON u.id = um.utilizador_id
GROUP BY m.nome
ORDER BY jogadores DESC;
```

```
-- 9. Aceitar o convite do Cristiano (UPDATE)
UPDATE convites
SET estado = 'aceite', respondido_em = NOW()
WHERE id = 2 AND destinatario_id = 1;

-- 10. O Tchali entra no jogo 1 (INSERT) — o UUID() gera o código do QR
INSERT INTO jogo_participantes (jogo_id, utilizador_id, codigo_checkin)
VALUES (1, 2, UUID());

-- 11. Desfazer os passos 9 e 10 (DELETE e UPDATE)
DELETE FROM jogo_participantes WHERE jogo_id = 1 AND utilizador_id = 2;
UPDATE convites SET estado = 'pendente', respondido_em = NULL WHERE id = 2;
```

Exercícios para a equipa

- [ ] Listar os jogos de Padel com preço até 10 €.
- [ ] Mostrar quantos convites cada utilizador enviou (`GROUP BY remetente_id`).
- [ ] Encontrar quem está livre ao sábado de manhã (`dia_semana = 6 AND hora_inicio < '12:00'`).
- [ ] Mudar o Tchali para Avançado em Padel e voltar a correr a consulta 4.

Parte 6 — Criar a API Node.js

Nesta parte montam o "esqueleto" da API: o projeto Node, as configurações, a ligação ao MySQL e o servidor. As rotas vêm nas partes 7 a 10.

6.1 Criar o projeto e instalar os pacotes

Na raiz do repositório:

```
mkdir -p backend/src/routes backend/src/middleware backend/src/utils
cd backend
npm init -y
npm install express mysql2 bcryptjs jsonwebtoken dotenv cors
```

Pacote	Para que serve
`express`	O servidor web: recebe pedidos HTTP e encaminha-os para as rotas
`mysql2`	Fala com o MySQL; a versão `mysql2/promise` permite usar `await`
`bcryptjs`	Cifra as palavras-passe (hash) e verifica-as no login

Pacote	Para que serve
`jsonwebtoken`	Cria e valida os tokens de sessão (JWT)
`dotenv`	Lê as configurações do ficheiro `.env`
`cors`	Deixa a app Flutter, a correr no browser, chamar a API noutra porta

6.2 `backend/package.json`

Substituam o que o `npm init` criou por isto. O script `dev` usa `node --watch`, que reinicia a API sozinho sempre que gravam um ficheiro.

```
{
  "name": "whoplay-api",
  "version": "1.0.0",
  "description": "API REST do WhoPlay (Node.js + Express + MySQL)",
  "main": "src/server.js",
  "type": "commonjs",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js"
  },
  "license": "ISC",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^18.0.9",
    "express": "^5.3.0",
    "jsonwebtoken": "^9.0.3",
    "mysql2": "^3.24.6"
  }
}
```

Se o `npm install` instalou versões um pouco diferentes, mantenham as vossas. O importante é o **Express 5**, que trata sozinho os erros das funções `async`.

6.3 Configurações: `.env.example`, `.env` e `.gitignore`

O `.env` guarda dados que mudam de PC para PC (e segredos). **Nunca vai para o GitHub.** Para isso existe o `.env.example`, que vai para o repositório como modelo: cada pessoa da equipa copia-o para `.env`.

`backend/.env.example`:

```
# Copiem este ficheiro para .env e ajustem os valores.
# O .env NÃO vai para o GitHub (está no .gitignore).

PORT=3000

# MySQL do XAMPP: utilizador root sem palavra-passe
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=whoplay

# Texto secreto usado para assinar os tokens de sessão (JWT).
# Usem um texto comprido e diferente em produção.
JWT_SECRET=troquem-isto-por-um-texto-secreto-comprido
JWT_EXPIRES_IN=7d
```

Criem o vosso `.env` e mudem o `JWT_SECRET` para um texto qualquer comprido:

```
cp .env.example .env
```

`backend/.gitignore`:

```
node_modules/
.env
```

6.4 `backend/src/config.js`

Lê o `.env` uma vez e junta tudo num objeto. Os outros ficheiros fazem `require('./config')` em vez de lerem `process.env` diretamente.

```
// Lê o ficheiro .env e junta todas as configurações num só sítio.
require('dotenv').config({ quiet: true });

module.exports = {
  port: Number(process.env.PORT) || 3000,
  db: {
    host: process.env.DB_HOST || 'localhost',
    port: Number(process.env.DB_PORT) || 3306,
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '',
    database: process.env.DB_NAME || 'whoplay',
  },
  jwtSecret: process.env.JWT_SECRET || 'so-para-desenvolvimento',
  jwtExpiresIn: process.env.JWT_EXPIRES_IN || '7d',
};
```

6.5 `backend/src/db.js` — ligação ao MySQL

Um *pool* mantém até 10 ligações abertas e reutiliza-as. Nas rotas, uma consulta faz-se assim: `const [linhas] = await db.query('SELECT ... WHERE id = ?', [id]);`.

Regra de ouro: os valores entram sempre pelos `?` e nunca colados no texto do SQL. O `mysql2` trata deles e isso impede ataques de *SQL injection*.

```
// Ligação ao MySQL. Um "pool" mantém várias ligações abertas e reutiliza-as.
const mysql = require('mysql2/promise');
const config = require('./config');

const pool = mysql.createPool({
  ...config.db,
  charset: 'utf8mb4',
  dateStrings: true, // datas chegam como texto "2026-10-31 10:00:00"
  decimalNumbers: true, // DECIMAL chega como número (8.00 → 8)
  connectionLimit: 10,
});

module.exports = pool;
```

6.6 `backend/src/utils/erros.js`

Nas rotas, quando algo está errado, lançam `throw new ErroApi(404, 'Mensagem')`. O servidor apanha o erro e responde `{ "erro": "Mensagem" }` com esse código HTTP. O `exigir` confirma os campos obrigatórios do pedido.

```
// Erro com código HTTP. Lançar `new ErroApi(404, 'Jogo não encontrado.')`
// numa rota faz a API responder { "erro": "Jogo não encontrado." } com 404.
class ErroApi extends Error {
  constructor(status, mensagem) {
    super(mensagem);
    this.status = status;
  }
}

// Confirma que o corpo do pedido tem todos os campos obrigatórios.
function exigir(corpo, campos) {
  const emFalta = campos.filter((c) => corpo[c] === undefined || corpo[c] ===
null || corpo[c] === '');
  if (emFalta.length > 0) {
    throw new ErroApi(400, `Campos obrigatórios em falta: ${emFalta.join(',
')}.`);
  }
}

module.exports = { ErroApi, exigir };
```

6.7 `backend/src/utils/datas.js`

```
// Ajudas para mostrar datas em português nos avisos.
const DIAS = ['domingo', 'segunda', 'terça', 'quarta', 'quinta', 'sexta',
'sábado'];

// "2026-10-31 10:00:00" → "sábado às 10:00"
function quando(dataHora) {
  const data = new Date(dataHora.replace(' ', 'T'));
  return `${DIAS[data.getDay()]} às ${dataHora.slice(11, 16)}`;
}

// Dia da semana de hoje no formato da BD: 1 = segunda … 7 = domingo
function diaDeHoje() {
  return ((new Date().getDay() + 6) % 7) + 1;
}

// Aceita "2026-10-31T10:00" ou "2026-10-31 10:00:00" e devolve o formato do
MySQL.
function paraMysql(dataHora) {
  const texto = String(dataHora).replace('T', ' ');
```

```
  return texto.length === 16 ? `${texto}:00` : texto.slice(0, 19);
}

module.exports = { quando, diaDeHoje, paraMysql };
```

6.8 `backend/src/server.js` — o servidor

Este é o ficheiro que arranca tudo. Pela ordem, faz o seguinte:

1. Ativa o `cors` e o `express.json()`.
2. Liga as rotas públicas (`/auth`, `/modalidades`) e as protegidas, que passam primeiro pelo `autenticar`.
3. Responde 404 às rotas que não existem.
4. Trata os erros num só sítio, e converte os erros do MySQL causados por dados errados (email repetido, id que não existe, horário inválido) em respostas 400 ou 409 com uma mensagem clara.

```
// Ponto de entrada da API: `npm run dev`
const express = require('express');
const cors = require('cors');
const config = require('./config');
const db = require('./db');
const autenticar = require('./middleware/auth');

const app = express();
app.use(cors()); // deixa a app Flutter (web) chamar a API
app.use(express.json()); // lê o corpo dos pedidos em JSON

// Rotas públicas
app.get('/', (req, res) => res.json({ app: 'WhoPlay API', estado: 'ok' }));
app.get('/saude', async (req, res) => {
  await db.query('SELECT 1');
  res.json({ api: 'ok', base_de_dados: 'ok' });
});
app.use('/auth', require('./routes/auth'));
app.use('/modalidades', require('./routes/modalidades'));

// Rotas protegidas (precisam de token)
app.use('/perfil', autenticar, require('./routes/perfil'));
app.use('/jogadores', autenticar, require('./routes/jogadores'));
app.use('/jogos', autenticar, require('./routes/jogos'));
app.use('/convites', autenticar, require('./routes/convites'));
app.use('/avisos', autenticar, require('./routes/avisos'));

// Rota que não existe → 404
```

```
app.use((req, res) => res.status(404).json({ erro: 'Rota não encontrada.'
}));

// Erros do MySQL que são culpa do pedido (dados errados) e não do servidor.
const ERROS_MYSQL = {
  ER_DUP_ENTRY: [409, 'Esse registo já existe.'],
  ER_NO_REFERENCED_ROW_2: [400, 'Um dos ids enviados não existe.'],
  ER_CHECK_CONSTRAINT_VIOLATED: [400, 'Valores fora das regras da base de
dados.'], // MySQL
  ER_CONSTRAINT_FAILED: [400, 'Valores fora das regras da base de dados.'],
// MariaDB
  WARN_DATA_TRUNCATED: [400, 'Um dos valores enviados não é válido.'],
  ER_TRUNCATED_WRONG_VALUE: [400, 'Um dos valores enviados não é válido.'],
};

// Qualquer erro lançado numa rota chega aqui.
app.use((erro, req, res, next) => {
  const conhecido = ERROS_MYSQL[erro.code];
  const status = erro.status || (conhecido ? conhecido[0] : 500);
  if (status === 500) console.error(erro);
  const mensagem = erro.status ? erro.message : conhecido ? conhecido[1] :
'Erro interno do servidor.';
  res.status(status).json({ erro: mensagem });
});

app.listen(config.port, () => {
  console.log(`WhoPlay API a correr em http://localhost:${config.port}`);
});
```

6.9 Arrancar a API

O `server.js` já aponta para as rotas das partes 7 a 10. Para o testarem antes de as escreverem, ponham `//` no início das linhas `require('./middleware/auth')` e `app.use(...)` cujos ficheiros ainda não existem, e tirem-no à medida que os criam.

```
npm run dev
```

No terminal deve aparecer `WhoPlay API a correr em http://localhost:3000`. Abram http://localhost:3000/saude no browser; a resposta deve ser `{"api":"ok","base_de_dados":"ok"}`. Se aparecer `Erro interno do servidor`, o MySQL do XAMPP está parado ou o `.env` está errado: vejam a mensagem no terminal.

Parte 7 — Autenticação: registo, login e JWT

O login devolve um **token JWT**: um texto assinado pela API que diz "este pedido vem do utilizador 1". A app guarda o token e envia-o em todos os pedidos seguintes, no cabeçalho `Authorization: Bearer <token>`. A API confirma a assinatura com o `JWT_SECRET` e fica a saber quem fez o pedido, sem voltar a pedir a palavra-passe.

1. **Registo ou login:** a app envia o email e a palavra-passe para `POST /auth/login`.
2. **A API verifica:** procura o email na tabela `utilizadores` e compara a palavra-passe com o `password_hash` usando o `bcrypt.compare`.
3. **A API responde** `{ token, utilizador }`. O token é válido durante 7 dias (`JWT_EXPIRES_IN`).
4. **Pedidos seguintes:** a app manda `Authorization: Bearer <token>`. O middleware `autenticar` valida-o e põe o id em `req.utilizadorId`.
5. **Terminar sessão (RF03):** a app esquece o token. Não é preciso nenhuma rota.

7.1 `backend/src/middleware/auth.js`

Um *middleware* é uma função que corre antes da rota. Se o token faltar ou for inválido, responde 401 e a rota nem chega a correr.

```
// Protege as rotas: só deixa passar pedidos com um token JWT válido
// no cabeçalho "Authorization: Bearer <token>".
const jwt = require('jsonwebtoken');
const config = require('../config');

function autenticar(req, res, next) {
  const cabecalho = req.headers.authorization || '';
  const token = cabecalho.startsWith('Bearer ') ? cabecalho.slice(7) : null;
  if (!token) {
    return res.status(401).json({ erro: 'Sessão em falta. Faz login.' });
  }
  try {
    const dados = jwt.verify(token, config.jwtSecret);
    req.utilizadorId = dados.id; // as rotas usam isto para saber quem é
    next();
  } catch {
    res.status(401).json({ erro: 'Sessão inválida ou expirada. Faz login
outra vez.' });
  }
}

module.exports = autenticar;
```

7.2 `backend/src/routes/auth.js`

Rota	Corpo do pedido	Resposta
`POST /auth/registo`	`{ "nome", "email", "password" }`	201 `{ token, utilizador }`; 409 se o email já existir; 400 se a palavra-passe tiver menos de 8 caracteres
`POST /auth/login`	`{ "email", "password" }`	200 `{ token, utilizador }`; 401 se os dados estiverem errados
`GET /auth/eu`	— (precisa de token)	`{ id, nome, email }`

No login, a mensagem de erro é a mesma para um email que não existe e para uma palavra-passe errada. Assim, quem tenta adivinhar não fica a saber que emails estão registados.

```
// Registo e login (RF01, RF02). O logout (RF03) é feito na app: basta apagar
o token.
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const db = require('../db');
const config = require('../config');
const autenticar = require('../middleware/auth');
const { ErroApi, exigir } = require('../utils/erros');

const router = express.Router();

function criarToken(id) {
  return jwt.sign({ id }, config.jwtSecret, { expiresIn: config.jwtExpiresIn
});
}

// POST /auth/registo  { nome, email, password }
router.post('/registo', async (req, res) => {
  exigir(req.body, ['nome', 'email', 'password']);
  const nome = String(req.body.nome).trim();
  const email = String(req.body.email).trim().toLowerCase();
  const password = String(req.body.password);
  if (password.length < 8) throw new ErroApi(400, 'A palavra-passe precisa de
pelo menos 8 caracteres.');

  const [existe] = await db.query('SELECT id FROM utilizadores WHERE email =
?', [email]);
  if (existe.length > 0) throw new ErroApi(409, 'Já existe uma conta com este
email.');
```

```
  const hash = await bcrypt.hash(password, 10);
  const [resultado] = await db.query(
    'INSERT INTO utilizadores (nome, email, password_hash) VALUES (?, ?, ?)',
    [nome, email, hash],
  );
  const id = resultado.insertId;
  res.status(201).json({ token: criarToken(id), utilizador: { id, nome, email
} });
});

// POST /auth/login  { email, password }
router.post('/login', async (req, res) => {
  exigir(req.body, ['email', 'password']);
  const email = String(req.body.email).trim().toLowerCase();
  const [linhas] = await db.query(
    'SELECT id, nome, email, password_hash FROM utilizadores WHERE email =
?',
    [email],
  );
  const utilizador = linhas[0];
  const certa = utilizador && (await
bcrypt.compare(String(req.body.password), utilizador.password_hash));
  if (!certa) throw new ErroApi(401, 'Email ou palavra-passe incorretos.');

  res.json({
    token: criarToken(utilizador.id),
    utilizador: { id: utilizador.id, nome: utilizador.nome, email:
utilizador.email },
  });
});

// GET /auth/eu  → quem está autenticado
router.get('/eu', autenticar, async (req, res) => {
  const [linhas] = await db.query('SELECT id, nome, email FROM utilizadores
WHERE id = ?', [req.utilizadorId]);
  if (linhas.length === 0) throw new ErroApi(404, 'Utilizador não
encontrado.');
  res.json(linhas[0]);
});

module.exports = router;
```

7.3 De onde veio o hash do `seed.sql`

Para gerarem o hash bcrypt de outra palavra-passe (por exemplo, para novos utilizadores no `seed.sql`), corram na pasta `backend/`:

```
node -e "console.log(require('bcryptjs').hashSync('12345678', 10))"
```

Cada vez sai um hash diferente (o bcrypt junta um valor aleatório, o *salt*), mas todos funcionam para a mesma palavra-passe.

Parte 8 — Perfil, modalidades e disponibilidade

Rota	Corpo do pedido	O que faz
`GET /modalidades`	— (pública)	Lista os 8 desportos (ecrã Onboarding)
`GET /perfil`	—	Devolve os dados do utilizador, os desportos e os horários (ecrã Perfil)
`PUT /perfil`	`{ nome?, localidade?, latitude?, longitude?, raio_km?, objetivo? }`	Atualiza só os campos enviados (RF04, RF08)
`PUT /perfil/modalidades`	`{ "modalidades": [{ "modalidade_id": 1, "nivel": "Intermédio" }] }`	Substitui a lista de desportos (RF05, RF06)
`PUT /perfil/disponibilidade`	`{ "disponibilidade": [{ "dia_semana": 1, "hora_inicio": "18:00", "hora_fim": "21:00" }] }`	Substitui os horários (RF07)

Transações. Para substituir uma lista, a API apaga a antiga e insere a nova. Se alguma inserção falhar a meio (um id que não existe, por exemplo), o utilizador ficaria sem desportos nenhuns. Para isso não acontecer, tudo corre numa **transação**: `beginTransaction()` → operações → `commit()`. Se houver um erro, o `rollback()` desfaz tudo e a lista antiga fica como estava.

`PUT /perfil` seguro. O SQL é montado só com os nomes da lista `permitidos`. Assim, ninguém consegue mudar o `email` ou o `password_hash` por esta rota, mesmo que os envie.

8.1 backend/src/routes/modalidades.js

```
// Lista de desportos (pública: o onboarding precisa dela antes do login
acabar).
const express = require('express');
const db = require('../db');

const router = express.Router();

// GET /modalidades
router.get('/', async (req, res) => {
  const [linhas] = await db.query('SELECT id, nome, descricao FROM
modalidades ORDER BY id');
  res.json(linhas);
});

module.exports = router;
```

8.2 backend/src/routes/perfil.js

```
// Perfil do utilizador autenticado (RF04–RF08).
const express = require('express');
const db = require('../db');
const { ErroApi, exigir } = require('../utils/erros');
const { NIVEIS } = require('../utils/compatibilidade');

const router = express.Router();

// GET /perfil  → dados, modalidades e disponibilidade
router.get('/', async (req, res) => {
  const id = req.utilizadorId;
  const [utilizadores] = await db.query(
    `SELECT id, nome, email, localidade, latitude, longitude, raio_km,
objetivo, avaliacao
     FROM utilizadores WHERE id = ?`,
    [id],
  );
  if (utilizadores.length === 0) throw new ErroApi(404, 'Utilizador não
encontrado.');

  const [modalidades] = await db.query(
    `SELECT m.id, m.nome, um.nivel
     FROM utilizador_modalidades um JOIN modalidades m ON m.id =
um.modalidade_id
     WHERE um.utilizador_id = ? ORDER BY m.id`,
    [id],
```

```
  );
  const [disponibilidade] = await db.query(
    `SELECT dia_semana, TIME_FORMAT(hora_inicio, '%H:%i') AS hora_inicio,
TIME_FORMAT(hora_fim, '%H:%i') AS hora_fim
     FROM disponibilidades WHERE utilizador_id = ? ORDER BY dia_semana,
hora_inicio`,
    [id],
  );
  res.json({ ...utilizadores[0], modalidades, disponibilidade });
});

// PUT /perfil  { nome?, localidade?, latitude?, longitude?, raio_km?,
objetivo? }
router.put('/', async (req, res) => {
  const permitidos = ['nome', 'localidade', 'latitude', 'longitude',
'raio_km', 'objetivo'];
  const campos = permitidos.filter((c) => req.body[c] !== undefined);
  if (campos.length === 0) throw new ErroApi(400, 'Nada para atualizar.');

  // Monta "nome = ?, localidade = ?" só com os campos enviados.
  const sets = campos.map((c) => `${c} = ?`).join(', ');
  const valores = campos.map((c) => req.body[c]);
  await db.query(`UPDATE utilizadores SET ${sets} WHERE id = ?`, [...valores,
req.utilizadorId]);
  res.json({ ok: true });
});

// PUT /perfil/modalidades  { modalidades: [{ modalidade_id, nivel }] }
// Substitui a lista inteira (usado no onboarding e no "Editar").
router.put('/modalidades', async (req, res) => {
  exigir(req.body, ['modalidades']);
  const lista = req.body.modalidades;
  if (!Array.isArray(lista)) throw new ErroApi(400, '"modalidades" tem de ser
uma lista.');
  for (const m of lista) {
    if (!NIVEIS.includes(m.nivel ?? 'Iniciante')) throw new ErroApi(400,
`Nível inválido: ${m.nivel}.`);
  }

  // Transação: ou corre tudo, ou não fica nada a meio.
  const ligacao = await db.getConnection();
  try {
    await ligacao.beginTransaction();
    await ligacao.query('DELETE FROM utilizador_modalidades WHERE
utilizador_id = ?', [req.utilizadorId]);
    for (const m of lista) {
```

```
      await ligacao.query(
        'INSERT INTO utilizador_modalidades (utilizador_id, modalidade_id,
nivel) VALUES (?, ?, ?)',
        [req.utilizadorId, m.modalidade_id, m.nivel ?? 'Iniciante'],
      );
    }
    await ligacao.commit();
  } catch (erro) {
    await ligacao.rollback();
    throw erro;
  } finally {
    ligacao.release();
  }
  res.json({ ok: true });
});

// PUT /perfil/disponibilidade  { disponibilidade: [{ dia_semana,
hora_inicio, hora_fim }] }
router.put('/disponibilidade', async (req, res) => {
  exigir(req.body, ['disponibilidade']);
  const lista = req.body.disponibilidade;
  if (!Array.isArray(lista)) throw new ErroApi(400, '"disponibilidade" tem de
ser uma lista.');
  for (const d of lista) {
    const dia = Number(d.dia_semana);
    if (!(dia >= 1 && dia <= 7)) throw new ErroApi(400, 'dia_semana tem de
ser de 1 (segunda) a 7 (domingo).');
    if (!(String(d.hora_fim) > String(d.hora_inicio))) {
      throw new ErroApi(400, 'A hora de fim tem de ser depois da hora de
início.');
    }
  }

  const ligacao = await db.getConnection();
  try {
    await ligacao.beginTransaction();
    await ligacao.query('DELETE FROM disponibilidades WHERE utilizador_id =
?', [req.utilizadorId]);
    for (const d of lista) {
      await ligacao.query(
        'INSERT INTO disponibilidades (utilizador_id, dia_semana,
hora_inicio, hora_fim) VALUES (?, ?, ?, ?)',
        [req.utilizadorId, d.dia_semana, d.hora_inicio, d.hora_fim],
      );
    }
    await ligacao.commit();
```

```
  } catch (erro) {
    await ligacao.rollback();
    throw erro;
  } finally {
    ligacao.release();
  }
  res.json({ ok: true });
});

module.exports = router;
```

O `perfil.js` usa a lista `NIVEIS` do `compatibilidade.js`, que fica criado na parte seguinte.

Parte 9 — Procurar jogadores e calcular a compatibilidade

A compatibilidade (RF10) vale de 0 a 100 pontos e soma 5 critérios, com os mesmos pesos do ecrã "Perfil do jogador". É calculada na API sempre que alguém procura jogadores.

Critério	Máximo	Como se calcula
Mesmo desporto	30	30 se os 2 jogam a modalidade escolhida
Nível	25	Mesmo nível = 25; 1 nível de diferença = 13; 2 níveis = 0
Disponibilidade	20	20 × (minutos livres em comum na semana ÷ os meus minutos livres)
Distância	15	15 × (1 − distância ÷ (2 × o meu raio)); 0 a partir do dobro do raio
Objetivo	10	Mesmo objetivo = 10; diferente = 4

Exemplo: Sebastião × Tchali, em Padel

Critério	Conta	Pontos
Mesmo desporto	Os 2 jogam Padel	30
Nível	Intermédio e Intermédio	25
Disponibilidade	Em comum: segunda 3 h + quarta 3 h + sábado 3,5 h = 9,5 h dos 12 h do Sebastião → 20 × 0,79	16
Distância	1,8 km com raio de 5 km → 15 × (1 − 1,8 ÷ 10)	12

Critério	Conta	Pontos
Objetivo	Os 2 querem "Jogos amigáveis"	10
Total		**93%**

Com o `seed.sql`, o resultado é Tchali 93%, Helio 85% e Cristiano 63%.

Estatística. A resposta de `GET /jogadores` traz também a **média** e o **desvio-padrão** das compatibilidades encontradas (80,3 e 12,7 com os dados de exemplo). O desvio-padrão mede o quanto os valores se afastam da média:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

9.1 `backend/src/utils/compatibilidade.js`

```
// Cálculo da compatibilidade entre 2 jogadores (RF10).
// Total máximo = 100 pontos, repartidos por 5 critérios.
const PESOS = { desporto: 30, nivel: 25, disponibilidade: 20, distancia: 15,
objetivo: 10 };
const NIVEIS = ['Iniciante', 'Intermédio', 'Avançado'];
const RAIO_TERRA_KM = 6371;

// Método numérico: fórmula de Haversine (distância sobre a superfície da
Terra).
function distanciaKm(lat1, lon1, lat2, lon2) {
  const rad = (graus) => (graus * Math.PI) / 180;
  const dLat = rad(lat2 - lat1);
  const dLon = rad(lon2 - lon1);
  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(rad(lat1)) * Math.cos(rad(lat2)) * Math.sin(dLon / 2) ** 2;
  return 2 * RAIO_TERRA_KM * Math.asin(Math.sqrt(a));
}

// "18:30:00" → 1110 minutos desde a meia-noite
function minutos(hora) {
  const [h, m] = hora.split(':').map(Number);
  return h * 60 + m;
}

// Minutos em que os 2 estão livres ao mesmo tempo, somados na semana
inteira.
function minutosEmComum(minhas, dele) {
  let total = 0;
  for (const a of minhas) {
```

```
    for (const b of dele) {
      if (a.dia_semana !== b.dia_semana) continue;
      const inicio = Math.max(minutos(a.hora_inicio),
minutos(b.hora_inicio));
      const fim = Math.min(minutos(a.hora_fim), minutos(b.hora_fim));
      total += Math.max(0, fim - inicio);
    }
  }
  return total;
}

// Devolve { total, criterios: [{ nome, pontos, maximo }] }.
function calcularCompatibilidade({ eu, outro, distancia, raioKm }) {
  // 1. Mesmo desporto: os 2 jogam a modalidade escolhida.
  const desporto = eu.nivel && outro.nivel ? PESOS.desporto : 0;

  // 2. Nível: igual = 25, 1 nível de diferença = 13, 2 níveis = 0.
  let nivel = 0;
  if (eu.nivel && outro.nivel) {
    const diferenca = Math.abs(NIVEIS.indexOf(eu.nivel) -
NIVEIS.indexOf(outro.nivel));
    nivel = PESOS.nivel * (1 - diferenca / 2);
  }

  // 3. Disponibilidade: parte do MEU tempo livre em que o outro também está
livre.
  const meusMinutos = eu.disponibilidade.reduce(
    (soma, d) => soma + minutos(d.hora_fim) - minutos(d.hora_inicio),
    0,
  );
  const disponibilidade =
    meusMinutos > 0
      ? PESOS.disponibilidade * (minutosEmComum(eu.disponibilidade,
outro.disponibilidade) / meusMinutos)
      : 0;

  // 4. Distância: 0 km = 15 pontos, desce até 0 ao dobro do raio.
  const distanciaPontos = PESOS.distancia * Math.max(0, 1 - distancia / (2 *
raioKm));

  // 5. Objetivo: igual = 10, diferente = 4.
  const objetivo = eu.objetivo === outro.objetivo ? PESOS.objetivo : 4;

  const criterios = [
    { nome: 'Mesmo desporto', pontos: Math.round(desporto), maximo:
PESOS.desporto },
```

```
    { nome: 'Nível', pontos: Math.round(nivel), maximo: PESOS.nivel },
    { nome: 'Disponibilidade', pontos: Math.round(disponibilidade), maximo:
PESOS.disponibilidade },
    { nome: 'Distância', pontos: Math.round(distanciaPontos), maximo:
PESOS.distancia },
    { nome: 'Objetivo', pontos: Math.round(objetivo), maximo: PESOS.objetivo
},
  ];
  const total = criterios.reduce((soma, c) => soma + c.pontos, 0);
  return { total, criterios };
}

// Estatística: média e desvio-padrão das compatibilidades encontradas.
function estatisticas(valores) {
  if (valores.length === 0) return { total: 0, media: 0, desvio_padrao: 0 };
  const media = valores.reduce((s, v) => s + v, 0) / valores.length;
  const variancia = valores.reduce((s, v) => s + (v - media) ** 2, 0) /
valores.length;
  return {
    total: valores.length,
    media: Math.round(media * 10) / 10,
    desvio_padrao: Math.round(Math.sqrt(variancia) * 10) / 10,
  };
}

module.exports = { distanciaKm, calcularCompatibilidade, estatisticas, NIVEIS
};
```

9.2 `backend/src/routes/jogadores.js`

Rota	O que devolve
`GET /jogadores?modalidade=1&raio=5`	`{ jogadores: [...], estatisticas: { total, media, desvio_padrao } }`, ordenados da maior para a menor compatibilidade. Sem `raio`, usa o `raio_km` do perfil
`GET /jogadores/2?modalidade=1`	Um jogador, com os 5 critérios (ecrã Perfil do jogador)

Cada jogador traz `id`, `nome`, `nivel`, `distancia_km`, `compatibilidade`, `criterios`, `avaliacao`, `objetivo` e `disponivel_hoje` (por exemplo, "19:00 – 21:00", ou `null`).

A função `carregarDisponibilidades` vai buscar os horários de todos os jogadores numa só consulta (`WHERE utilizador_id IN (?)`), em vez de fazer uma consulta por jogador. Com muitos jogadores, isto faz uma grande diferença.

```
// Procurar jogadores compatíveis (RF09, RF10).
const express = require('express');
const db = require('../db');
const { ErroApi } = require('../utils/erros');
const { diaDeHoje } = require('../utils/datas');
const { distanciaKm, calcularCompatibilidade, estatisticas } =
require('../utils/compatibilidade');

const router = express.Router();

// Disponibilidades de vários utilizadores de uma vez → Map(id → lista)
async function carregarDisponibilidades(ids) {
  const [linhas] = await db.query(
    `SELECT utilizador_id, dia_semana,
            TIME_FORMAT(hora_inicio, '%H:%i') AS hora_inicio,
TIME_FORMAT(hora_fim, '%H:%i') AS hora_fim
     FROM disponibilidades WHERE utilizador_id IN (?) ORDER BY hora_inicio`,
    [ids],
  );
  const mapa = new Map(ids.map((id) => [id, []]));
  for (const linha of linhas) mapa.get(linha.utilizador_id).push(linha);
  return mapa;
}

// Compara o utilizador autenticado com os outros jogadores de uma
modalidade.
// Com `apenasId`, devolve só esse jogador (e não filtra pelo raio).
async function avaliarJogadores(meuId, modalidadeId, raioPedido, apenasId =
null) {
  const [eus] = await db.query(
    `SELECT u.id, u.latitude, u.longitude, u.raio_km, u.objetivo, um.nivel
     FROM utilizadores u
     LEFT JOIN utilizador_modalidades um ON um.utilizador_id = u.id AND
um.modalidade_id = ?
     WHERE u.id = ?`,
    [modalidadeId, meuId],
  );
  const eu = eus[0];
  if (!eu) throw new ErroApi(404, 'Utilizador não encontrado.');
  if (eu.latitude === null) throw new ErroApi(400, 'Define primeiro a tua
localização no perfil.');
  const raioKm = Number(raioPedido) || eu.raio_km;
```

```
  const parametros = [modalidadeId, meuId];
  if (apenasId) parametros.push(apenasId);
  const [outros] = await db.query(
    `SELECT u.id, u.nome, u.latitude, u.longitude, u.objetivo, u.avaliacao,
um.nivel, m.nome AS modalidade
     FROM utilizadores u
     JOIN utilizador_modalidades um ON um.utilizador_id = u.id AND
um.modalidade_id = ?
     JOIN modalidades m ON m.id = um.modalidade_id
     WHERE u.id <> ? AND u.latitude IS NOT NULL ${apenasId ? 'AND u.id = ?' :
''}`,
    parametros,
  );

  const disponibilidades = await carregarDisponibilidades([meuId,
...outros.map((o) => o.id)]);
  eu.disponibilidade = disponibilidades.get(meuId);
  const hoje = diaDeHoje();

  const resultado = [];
  for (const outro of outros) {
    const distancia = distanciaKm(eu.latitude, eu.longitude, outro.latitude,
outro.longitude);
    if (!apenasId && distancia > raioKm) continue;

    outro.disponibilidade = disponibilidades.get(outro.id);
    const { total, criterios } = calcularCompatibilidade({ eu, outro,
distancia, raioKm });
    const hojeLivre = outro.disponibilidade.find((d) => d.dia_semana ===
hoje);

    resultado.push({
      id: outro.id,
      nome: outro.nome,
      modalidade: outro.modalidade,
      nivel: outro.nivel,
      distancia_km: Math.round(distancia * 10) / 10,
      compatibilidade: total,
      criterios,
      avaliacao: outro.avaliacao,
      objetivo: outro.objetivo,
      disponivel_hoje: hojeLivre ? `${hojeLivre.hora_inicio} –
${hojeLivre.hora_fim}` : null,
    });
  }
```

```
  resultado.sort((a, b) => b.compatibilidade - a.compatibilidade);
  return resultado;
}

// GET /jogadores?modalidade=1&raio=5
router.get('/', async (req, res) => {
  const modalidade = Number(req.query.modalidade) || 1;
  const jogadores = await avaliarJogadores(req.utilizadorId, modalidade,
req.query.raio);
  res.json({
    jogadores,
    estatisticas: estatisticas(jogadores.map((j) => j.compatibilidade)),
  });
});

// GET /jogadores/2?modalidade=1
router.get('/:id', async (req, res) => {
  const modalidade = Number(req.query.modalidade) || 1;
  const [jogador] = await avaliarJogadores(req.utilizadorId, modalidade,
null, Number(req.params.id));
  if (!jogador) throw new ErroApi(404, 'Jogador não encontrado nesta
modalidade.');
  res.json(jogador);
});

module.exports = router;
```

Parte 10 — Jogos, convites, avisos e check-in

As últimas 3 rotas da API servem os ecrãs Criar jogo, Mapa, Detalhe do jogo, Check-in e Convites. Sempre que alguém entra num jogo ou responde a um convite, a API cria um aviso para a outra pessoa na tabela `notificacoes`.

Rota	Corpo do pedido	O que faz
`GET /jogos?modalidade=1`	—	Jogos abertos e futuros, com `vagas` e `distancia_km`
`GET /jogos/1`	—	Detalhe + `participantes` + `eu_participo`

Rota	Corpo do pedido	O que faz
`POST /jogos`	`{ modalidade_id, data_hora, local_nome, nivel, formato, capacidade, vagas_abertas, preco }`	Cria o jogo e mete o organizador como participante
`POST /jogos/1/entrar`	—	Ocupa uma vaga e devolve `{ codigo_checkin }`; se já lá estiver, devolve o mesmo código
`GET /jogos/1/checkin`	—	O meu código QR para esse jogo
`POST /jogos/checkin`	`{ codigo }`	O organizador lê o QR de alguém e confirma a presença
`GET /convites`	—	`{ enviados, recebidos }` por responder
`POST /convites`	`{ destinatario_id, modalidade_id, data_hora }`	Envia um convite e avisa o destinatário
`POST /convites/2/aceitar` e `/recusar`	—	Responde a um convite e avisa quem o enviou
`GET /avisos`	—	Os 30 avisos mais recentes
`POST /avisos/lidos` e `POST /avisos/5/lido`	—	Marca todos, ou um, como lidos

A última vaga. Se 2 pessoas carregarem em "Entrar no jogo" ao mesmo tempo, as 2 podiam ver "1 vaga" e entrar ambas. Para o evitar, o `SELECT ... FOR UPDATE` bloqueia a linha do jogo até ao fim da transação: a segunda pessoa espera, vê 0 vagas e recebe o erro 409 "Já não há vagas neste jogo.".

O código do QR é um UUID gerado pelo `crypto.randomUUID()` do Node, por exemplo `a55a3d90-fd03-4731-a648-f2579950c3bd`. É impossível de adivinhar, e por isso serve de "bilhete".

10.1 backend/src/routes/jogos.js

```
// Criar jogos, entrar em jogos e check-in por QR Code.
const crypto = require('node:crypto');
const express = require('express');
const db = require('../db');
const { ErroApi, exigir } = require('../utils/erros');
const { quando, paraMysql } = require('../utils/datas');
const { distanciaKm } = require('../utils/compatibilidade');

const router = express.Router();

// Base comum: dados do jogo + quantos lugares já estão ocupados.
const SELECT_JOGO = `
  SELECT j.id, j.organizador_id, o.nome AS organizador, j.modalidade_id,
m.nome AS modalidade,
         j.data_hora, j.local_nome, j.latitude, j.longitude, j.nivel,
j.formato,
         j.capacidade, j.preco, j.estado,
         j.jogadores_externos + (SELECT COUNT(*) FROM jogo_participantes p
WHERE p.jogo_id = j.id) AS ocupados
  FROM jogos j
  JOIN modalidades m  ON m.id = j.modalidade_id
  JOIN utilizadores o ON o.id = j.organizador_id`;

async function minhaPosicao(id) {
  const [linhas] = await db.query('SELECT nome, latitude, longitude FROM
utilizadores WHERE id = ?', [id]);
  return linhas[0];
}

// Acrescenta as vagas e a distância a que o jogo fica de mim.
function completar(jogo, eu) {
  const distancia =
    eu.latitude !== null && jogo.latitude !== null
      ? Math.round(distanciaKm(eu.latitude, eu.longitude, jogo.latitude,
jogo.longitude) * 10) / 10
      : null;
  return { ...jogo, vagas: jogo.capacidade - jogo.ocupados, distancia_km:
distancia };
}

// GET /jogos?modalidade=1  → jogos abertos e futuros
router.get('/', async (req, res) => {
  const modalidade = Number(req.query.modalidade) || null;
  const [jogos] = await db.query(
```

```
    `${SELECT_JOGO}
     WHERE j.estado = 'aberto' AND j.data_hora > NOW() ${modalidade ? 'AND
j.modalidade_id = ?' : ''}
     ORDER BY j.data_hora`,
    modalidade ? [modalidade] : [],
  );
  const eu = await minhaPosicao(req.utilizadorId);
  res.json(jogos.map((j) => completar(j, eu)));
});

// POST /jogos/checkin  { codigo }  → o organizador lê o QR e confirma a
presença
router.post('/checkin', async (req, res) => {
  exigir(req.body, ['codigo']);
  const [linhas] = await db.query(
    `SELECT p.jogo_id, p.utilizador_id, p.checkin_em, j.organizador_id,
u.nome
     FROM jogo_participantes p
     JOIN jogos j        ON j.id = p.jogo_id
     JOIN utilizadores u ON u.id = p.utilizador_id
     WHERE p.codigo_checkin = ?`,
    [req.body.codigo],
  );
  const participante = linhas[0];
  if (!participante) throw new ErroApi(404, 'QR Code desconhecido.');
  if (participante.organizador_id !== req.utilizadorId) {
    throw new ErroApi(403, 'Só o organizador pode confirmar presenças.');
  }
  await db.query(
    `UPDATE jogo_participantes SET checkin_em = COALESCE(checkin_em, NOW())
     WHERE jogo_id = ? AND utilizador_id = ?`,
    [participante.jogo_id, participante.utilizador_id],
  );
  res.json({ ok: true, jogo_id: participante.jogo_id, nome: participante.nome
});
});

// GET /jogos/1  → detalhe + quem vai
router.get('/:id', async (req, res) => {
  const [jogos] = await db.query(`${SELECT_JOGO} WHERE j.id = ?`,
[req.params.id]);
  if (jogos.length === 0) throw new ErroApi(404, 'Jogo não encontrado.');

  const [participantes] = await db.query(
    `SELECT u.id, u.nome, (u.id = j.organizador_id) AS organizador, um.nivel,
p.checkin_em
```

```
     FROM jogo_participantes p
     JOIN jogos j        ON j.id = p.jogo_id
     JOIN utilizadores u ON u.id = p.utilizador_id
     LEFT JOIN utilizador_modalidades um ON um.utilizador_id = u.id AND
um.modalidade_id = j.modalidade_id
     WHERE p.jogo_id = ?
     ORDER BY organizador DESC, p.entrou_em, u.id`,
    [req.params.id],
  );

  const eu = await minhaPosicao(req.utilizadorId);
  res.json({
    ...completar(jogos[0], eu),
    participantes: participantes.map((p) => ({ ...p, organizador:
Boolean(p.organizador) })),
    eu_participo: participantes.some((p) => p.id === req.utilizadorId),
  });
});

// POST /jogos  { modalidade_id, data_hora, local_nome, nivel, formato,
capacidade, vagas_abertas, preco }
router.post('/', async (req, res) => {
  exigir(req.body, ['modalidade_id', 'data_hora', 'local_nome', 'nivel',
'capacidade', 'vagas_abertas']);
  const capacidade = Number(req.body.capacidade);
  const vagas = Number(req.body.vagas_abertas);
  // O organizador ocupa 1 lugar; o resto que não está em vagas são pessoas
de fora da app.
  const externos = capacidade - vagas - 1;
  if (vagas < 1 || externos < 0) throw new ErroApi(400, 'Número de vagas
inválido para esta capacidade.');

  const eu = await minhaPosicao(req.utilizadorId);
  const ligacao = await db.getConnection();
  let jogoId;
  try {
    await ligacao.beginTransaction();
    const [resultado] = await ligacao.query(
      `INSERT INTO jogos (organizador_id, modalidade_id, data_hora,
local_nome, latitude, longitude,
                          nivel, formato, capacidade, jogadores_externos,
preco)
       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
      [
        req.utilizadorId,
        req.body.modalidade_id,
```

```
        paraMysql(req.body.data_hora),
        req.body.local_nome,
        req.body.latitude ?? eu.latitude, // sem coordenadas do local, usa as
do organizador
        req.body.longitude ?? eu.longitude,
        req.body.nivel,
        req.body.formato ?? 'Amigável',
        capacidade,
        externos,
        Number(req.body.preco) || 0,
      ],
    );
    jogoId = resultado.insertId;
    await ligacao.query(
      'INSERT INTO jogo_participantes (jogo_id, utilizador_id,
codigo_checkin) VALUES (?, ?, ?)',
      [jogoId, req.utilizadorId, crypto.randomUUID()],
    );
    await ligacao.commit();
  } catch (erro) {
    await ligacao.rollback();
    throw erro;
  } finally {
    ligacao.release();
  }
  res.status(201).json({ id: jogoId });
});

// POST /jogos/1/entrar  → ocupa uma vaga e devolve o código do QR
router.post('/:id/entrar', async (req, res) => {
  const jogoId = Number(req.params.id);
  const ligacao = await db.getConnection();
  try {
    await ligacao.beginTransaction();
    // FOR UPDATE bloqueia o jogo até ao fim da transação: 2 pessoas não
ficam com a última vaga.
    const [jogos] = await ligacao.query(
      `SELECT j.id, j.organizador_id, j.estado, j.capacidade, j.data_hora,
m.nome AS modalidade,
              j.jogadores_externos + (SELECT COUNT(*) FROM jogo_participantes
p WHERE p.jogo_id = j.id) AS ocupados
       FROM jogos j JOIN modalidades m ON m.id = j.modalidade_id
       WHERE j.id = ? FOR UPDATE`,
      [jogoId],
    );
    const jogo = jogos[0];
```

```
    if (!jogo) throw new ErroApi(404, 'Jogo não encontrado.');

    const [jaEstou] = await ligacao.query(
      'SELECT codigo_checkin FROM jogo_participantes WHERE jogo_id = ? AND
utilizador_id = ?',
      [jogoId, req.utilizadorId],
    );
    if (jaEstou.length > 0) {
      await ligacao.commit();
      return res.json({ codigo_checkin: jaEstou[0].codigo_checkin });
    }
    if (jogo.estado !== 'aberto') throw new ErroApi(409, 'Este jogo já não
está aberto.');
    if (jogo.ocupados >= jogo.capacidade) throw new ErroApi(409, 'Já não há
vagas neste jogo.');

    const codigo = crypto.randomUUID();
    await ligacao.query(
      'INSERT INTO jogo_participantes (jogo_id, utilizador_id,
codigo_checkin) VALUES (?, ?, ?)',
      [jogoId, req.utilizadorId, codigo],
    );
    const [eu] = await ligacao.query('SELECT nome FROM utilizadores WHERE id
= ?', [req.utilizadorId]);
    await ligacao.query(
      'INSERT INTO notificacoes (utilizador_id, titulo, detalhe) VALUES (?,
?, ?)',
      [jogo.organizador_id, `${eu[0].nome} entrou no teu jogo`,
`${jogo.modalidade} · ${quando(jogo.data_hora)}`],
    );
    await ligacao.commit();
    res.status(201).json({ codigo_checkin: codigo });
  } catch (erro) {
    await ligacao.rollback();
    throw erro;
  } finally {
    ligacao.release();
  }
});

// GET /jogos/1/checkin  → o meu código QR para este jogo
router.get('/:id/checkin', async (req, res) => {
  const [linhas] = await db.query(
    'SELECT codigo_checkin, checkin_em FROM jogo_participantes WHERE jogo_id
= ? AND utilizador_id = ?',
    [req.params.id, req.utilizadorId],
```

```
  );
  if (linhas.length === 0) throw new ErroApi(404, 'Ainda não entraste neste
jogo.');
  res.json(linhas[0]);
});

module.exports = router;
```

A rota `POST /checkin` está escrita **antes** de `GET /:id`. No Express, as rotas são verificadas por ordem, e assim não há confusão entre a palavra `checkin` e um id.

10.2 `backend/src/routes/convites.js`

```
// Convites entre jogadores (ecrã Convites).
const express = require('express');
const db = require('../db');
const { ErroApi, exigir } = require('../utils/erros');
const { quando, paraMysql } = require('../utils/datas');
const { distanciaKm } = require('../utils/compatibilidade');

const router = express.Router();

// GET /convites  → { enviados: [...], recebidos: [...] } (só os que estão
por responder)
router.get('/', async (req, res) => {
  const id = req.utilizadorId;
  const [enviados] = await db.query(
    `SELECT c.id, c.estado, c.data_hora, m.nome AS modalidade, u.id AS
jogador_id, u.nome
     FROM convites c
     JOIN utilizadores u ON u.id = c.destinatario_id
     JOIN modalidades m  ON m.id = c.modalidade_id
     WHERE c.remetente_id = ? AND c.estado = 'pendente'
     ORDER BY c.data_hora`,
    [id],
  );
  const [recebidos] = await db.query(
    `SELECT c.id, c.estado, c.data_hora, m.nome AS modalidade, u.id AS
jogador_id, u.nome,
            u.latitude, u.longitude, eu.latitude AS minha_lat, eu.longitude
AS minha_lon
     FROM convites c
     JOIN utilizadores u  ON u.id = c.remetente_id
     JOIN utilizadores eu ON eu.id = c.destinatario_id
     JOIN modalidades m   ON m.id = c.modalidade_id
     WHERE c.destinatario_id = ? AND c.estado = 'pendente'
```

```
     ORDER BY c.data_hora`,
    [id],
  );
  res.json({
    enviados,
    recebidos: recebidos.map(({ latitude, longitude, minha_lat, minha_lon,
...convite }) => ({
      ...convite,
      distancia_km:
        latitude !== null && minha_lat !== null
          ? Math.round(distanciaKm(minha_lat, minha_lon, latitude, longitude)
* 10) / 10
          : null,
    })),
  });
});

// POST /convites  { destinatario_id, modalidade_id, data_hora }
router.post('/', async (req, res) => {
  exigir(req.body, ['destinatario_id', 'modalidade_id', 'data_hora']);
  const destinatario = Number(req.body.destinatario_id);
  if (destinatario === req.utilizadorId) throw new ErroApi(400, 'Não te podes
convidar a ti próprio.');

  const [pessoas] = await db.query('SELECT id, nome FROM utilizadores WHERE
id IN (?, ?)', [
    req.utilizadorId,
    destinatario,
  ]);
  const eu = pessoas.find((p) => p.id === req.utilizadorId);
  if (!pessoas.some((p) => p.id === destinatario)) throw new ErroApi(404,
'Jogador não encontrado.');
  const [modalidades] = await db.query('SELECT nome FROM modalidades WHERE id
= ?', [req.body.modalidade_id]);
  if (modalidades.length === 0) throw new ErroApi(404, 'Modalidade não
encontrada.');

  const dataHora = paraMysql(req.body.data_hora);
  const [resultado] = await db.query(
    'INSERT INTO convites (remetente_id, destinatario_id, modalidade_id,
data_hora) VALUES (?, ?, ?, ?)',
    [req.utilizadorId, destinatario, req.body.modalidade_id, dataHora],
  );
  await db.query('INSERT INTO notificacoes (utilizador_id, titulo, detalhe)
VALUES (?, ?, ?)', [
    destinatario,
```

```
    `${eu.nome} convidou-te para jogar`,
    `${modalidades[0].nome} · ${quando(dataHora)}`,
  ]);
  res.status(201).json({ id: resultado.insertId });
});

// Aceitar ou recusar um convite recebido.
async function responder(req, res, estado) {
  const [convites] = await db.query(
    `SELECT c.id, c.remetente_id, c.estado, u.nome AS eu
     FROM convites c JOIN utilizadores u ON u.id = c.destinatario_id
     WHERE c.id = ? AND c.destinatario_id = ?`,
    [req.params.id, req.utilizadorId],
  );
  const convite = convites[0];
  if (!convite) throw new ErroApi(404, 'Convite não encontrado.');
  if (convite.estado !== 'pendente') throw new ErroApi(409, 'Este convite já
foi respondido.');

  await db.query('UPDATE convites SET estado = ?, respondido_em = NOW() WHERE
id = ?', [estado, convite.id]);
  const verbo = estado === 'aceite' ? 'aceitou' : 'recusou';
  await db.query('INSERT INTO notificacoes (utilizador_id, titulo) VALUES (?,
?)', [
    convite.remetente_id,
    `O ${convite.eu} ${verbo} o teu convite`,
  ]);
  res.json({ ok: true, estado });
}

// POST /convites/2/aceitar  e  POST /convites/2/recusar
router.post('/:id/aceitar', (req, res) => responder(req, res, 'aceite'));
router.post('/:id/recusar', (req, res) => responder(req, res, 'recusado'));

module.exports = router;
```

Reparem que a consulta de `responder` filtra por `c.destinatario_id = ?` com o id de quem fez o pedido. É isto que impede alguém de aceitar um convite que não é seu: para essa pessoa, o convite simplesmente "não existe" (404).

10.3 `backend/src/routes/avisos.js`

```
// Avisos (notificações dentro da app).
const express = require('express');
const db = require('../db');
```

```
const router = express.Router();

// GET /avisos  → os 30 mais recentes
router.get('/', async (req, res) => {
  const [avisos] = await db.query(
    `SELECT id, titulo, detalhe, lida, criado_em
     FROM notificacoes WHERE utilizador_id = ?
     ORDER BY criado_em DESC LIMIT 30`,
    [req.utilizadorId],
  );
  res.json(avisos.map((a) => ({ ...a, lida: Boolean(a.lida) })));
});

// POST /avisos/lidos  → marca todos como lidos
router.post('/lidos', async (req, res) => {
  await db.query('UPDATE notificacoes SET lida = TRUE WHERE utilizador_id =
?', [req.utilizadorId]);
  res.json({ ok: true });
});

// POST /avisos/5/lido  → marca um como lido
router.post('/:id/lido', async (req, res) => {
  await db.query('UPDATE notificacoes SET lida = TRUE WHERE id = ? AND
utilizador_id = ?', [
    req.params.id,
    req.utilizadorId,
  ]);
  res.json({ ok: true });
});

module.exports = router;
```

Com os 14 ficheiros criados, tirem os `//` que puseram no `server.js` (Parte 6.9). O `npm run dev` deve arrancar sem erros.

Parte 11 — Testar a API com o Thunder Client

Antes de ligarem a app, confirmem que cada rota responde como deve. O Thunder Client é uma extensão do VS Code que faz pedidos HTTP sem precisar de código.

11.1 Preparar

1. No VS Code: **Extensions** (Ctrl+Shift+X) → procurem **Thunder Client** → **Install**.
2. Confirmem que o MySQL do XAMPP e a API (`npm run dev`) estão a correr.

3. No ícone do Thunder Client (barra lateral) → **New Request**.

11.2 Fazer login e guardar o token

1. Método **POST**, URL `http://localhost:3000/auth/login`.
2. Separador **Body** → **JSON**:

```
{ "email": "sebastiao@email.com", "password": "12345678" }
```

3. **Send.** A resposta (200) traz um `token` comprido. Copiem-no sem as aspas.
4. Nos pedidos seguintes: separador **Auth** → **Bearer** → colem o token.

11.3 Sequência de testes

Corram por esta ordem, logo a seguir a recarregarem o `seed.sql` (Parte 4). Todos foram testados com estes resultados.

#	Pedido	Corpo (JSON)	Resposta esperada
1	`GET /saude`	—	200 `{"api":"ok","base_de_dados":"ok"}`
2	`GET /modalidades`	—	200, lista com 8 desportos
3	`POST /auth/login` (palavra-passe errada)	`{"email":"sebastiao@email.com","password":"x"}`	401 "Email ou palavra-passe incorretos."
4	`GET /perfil` sem token	—	401 "Sessão em falta. Faz login."
5	`GET /perfil`	—	200, Sebastião com 2 desportos e 3 horários
6	`GET /jogadores?modalidade=1`	—	200, Tchali 93, Helio 85, Cristiano 63; média 80,3
7	`GET /jogadores/2?modalidade=1`	—	200, Tchali com os 5 critérios
8	`GET /jogos`	—	200, Padel Club X, `vagas: 1`, `distancia_km: 2.4`
9	`GET /jogos/1`	—	200, 3 participantes, `eu_participo: true`
10	`GET /convites`	—	200, 1 enviado (Tchali) e 1 recebido (Cristiano)
11	`POST /convites/2/aceitar`	—	200 `{"ok":true,"estado":"aceite"}`
12	`POST /convites/2/aceitar` (outra vez)	—	409 "Este convite já foi respondido."
13	`POST /convites`	`{"destinatario_id":2,"modalidade_id":1,"data_hora":"2026-11-02 19:00"}`	201 `{"id":4}`
14	`POST /convites` a si próprio	`{"destinatario_id":1,"modalidade_id":1,"data_hora":"2026-11-02 19:00"}`	400 "Não te podes convidar a ti próprio."

#	Pedido	Corpo (JSON)	Resposta esperada
15	`POST /jogos`	`{"modalidade_id":1,"data_hora":"2026-11-07T10:00","local_nome":"Padel Club Y","nivel":"Intermédio","formato":"Amigável","capacidade":4,"vagas_abertas":2,"preco":7.5}`	201 `{"id":2}`
16	`PUT /perfil/disponibilidade` com horário ao contrário	`{"disponibilidade":[{"dia_semana":3,"hora_inicio":"21:00","hora_fim":"18:00"}]}`	400 "A hora de fim tem de ser depois da hora de início."
17	`GET /avisos`	—	200, avisos do mais recente para o mais antigo
18	`GET /nada`	—	404 "Rota não encontrada."

11.4 Testar o registo e a última vaga

1. `POST /auth/registo` com `{"nome":"Teste","email":"teste@email.com","password":"12345678"}` → 201 com um token novo. Usem esse token nos passos seguintes.
2. `GET /jogadores?modalidade=1` → 400 "Define primeiro a tua localização no perfil.".
3. `PUT /perfil` com `{"localidade":"Oriente","latitude":38.768,"longitude":-9.099}` → 200.
4. `POST /jogos/1/entrar` → 201 com o `codigo_checkin`. Era a última vaga.
5. Façam login como **tchali@email.com** e tentem `POST /jogos/1/entrar` → 409 "Já não há vagas neste jogo.".
6. Como **sebastiao@email.com** (organizador): `POST /jogos/checkin` com `{"codigo":"<o código do passo 4>"}` → 200 `{"ok":true,"jogo_id":1,"nome":"Teste"}`.

No fim, recarreguem os 2 scripts SQL para voltar ao estado inicial.

11.5 Testar com curl (alternativa)

Se preferirem a linha de comandos, os mesmos testes fazem-se com `curl` no Git Bash:

```
curl -s -X POST http://localhost:3000/auth/login -H "Content-Type: application/json" -d '{"email":"sebastiao@email.com","password":"12345678"}'
curl -s http://localhost:3000/jogadores?modalidade=1 -H "Authorization: Bearer COLEM_AQUI_O_TOKEN"
```

Atenção: no Windows, o `curl` estraga os acentos escritos diretamente no comando ("Intermédio" chega como "Interm□dio"). Para pedidos com acentos usem o Thunder Client, ou ponham o JSON num ficheiro e enviem-no com `--data-binary @ficheiro.json`.

Parte 12 — Ligar a app Flutter à API

A app deixa de usar o `mock_data.dart` e passa a pedir tudo à API. O aspeto dos ecrãs fica igual ao mockup; só muda a origem dos dados.

Ficheiro	Estado	O que faz
`lib/services/api.dart`	Novo	Todas as chamadas HTTP, o token e os erros
`lib/data/models.dart`	Novo, substitui o `mock_data.dart`	Classes Dart criadas a partir do JSON (`fromJson`)
`lib/utils/formatos.dart`	Novo	"Sábado, 31 de outubro às 10:00", "1,8 km", "8 €"…
`lib/widgets/async_view.dart`	Novo	Mostra "a carregar", o erro (com "Tentar outra vez") ou o conteúdo
`lib/screens/register_screen.dart`	Novo	Formulário "Criar conta" (nome, email, palavra-passe)
11 ecrãs em `lib/screens/`	Atualizados	Login, Onboarding, Perfil, Jogadores, Perfil do jogador, Convites, Criar jogo, Mapa, Detalhe, Check-in, HomeShell
`lib/data/mock_data.dart`	Apagar	Já não é usado

12.1 O padrão usado em todos os ecrãs

Os ecrãs que mostram dados seguem sempre os mesmos 3 passos:

```
// 1. Pedir os dados quando o ecrã abre (guardar o Future no estado)
late Future<Perfil> _perfil = Api.perfil();

// 2. Para atualizar, criar um Future novo dentro de setState
void _recarregar() => setState(() => _perfil = Api.perfil());

// 3. Desenhar com o AsyncView: a carregar → erro → conteúdo
AsyncView<Perfil>(
  future: _perfil,
  onRetry: _recarregar,
  builder: (context, perfil) => Text(perfil.nome),
)
```

Nas ações (botões), o padrão é `try` / `on ApiException` / `finally`: a mensagem de erro da API ("Já não há vagas neste jogo.") aparece numa *SnackBar*, e o botão fica desativado enquanto o pedido está a decorrer.

12.2 Acrescentar o pacote `http`

No `pubspec.yaml`, juntem esta linha em `dependencies` (a seguir ao `flutter_svg`):

```
  http: ^1.2.2
```

Depois:

```
flutter pub get
```

12.3 Que endereço usar para a API

O `localhost` é sempre “o próprio aparelho”. Por isso o endereço muda conforme onde a app corre:

A app corre em…	Endereço da API	Como
Chrome / Windows	`http://localhost:3000`	Automático
Emulador Android	`http://10.0.2.2:3000`	Automático (o 10.0.2.2 é o PC visto de dentro do emulador)
Telemóvel real (mesma rede Wi-Fi que o PC)	`http://<IP do PC>:3000`	`flutter run --dart-define=API_URL=http://192.168.1.10:3000` (o IP aparece com `ipconfig`)

No Android, os pedidos `http://` (sem "s") estão bloqueados por defeito em versões de produção. Para testar num telemóvel real, acrescentem `android:usesCleartextTraffic="true"` à etiqueta `<application>` de `android/app/src/main/AndroidManifest.xml`.

12.4 `lib/services/api.dart`

Todas as chamadas à API passam por esta classe. O `_pedido` junta o token, converte para JSON e transforma qualquer erro (`{"erro": "..."}`, servidor desligado, demora de mais de 10 segundos) numa `ApiException` com uma mensagem em português. O token fica guardado em memória: ao fechar a app é preciso voltar a fazer login. Para o manter guardado, podem usar mais tarde o pacote `shared_preferences`.

```
import 'dart:async';
import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;

import '../data/models.dart';
import '../utils/formatos.dart';

/// Erro devolvido pela API ({"erro": "..."}) ou falha de ligação.
class ApiException implements Exception {
  const ApiException(this.message, [this.status]);

  final String message;
  final int? status;

  @override
```

```
  String toString() => message;
}

/// Todas as chamadas à API do WhoPlay passam por aqui.
abstract final class Api {
  /// Para um telemóvel real: flutter run --dart-
define=API_URL=http://192.168.1.10:3000
  static const _urlDefinido = String.fromEnvironment('API_URL');

  static String get baseUrl {
    if (_urlDefinido.isNotEmpty) return _urlDefinido;
    // O emulador Android vê o PC no endereço 10.0.2.2 (localhost seria o
próprio emulador).
    if (!kIsWeb && defaultTargetPlatform == TargetPlatform.android) return
'http://10.0.2.2:3000';
    return 'http://localhost:3000';
  }

  static String? _token;

  static bool get autenticado => _token != null;

  /// RF03 — terminar sessão: basta esquecer o token.
  static void terminarSessao() => _token = null;

  // ---------------------------------------------------------------- base

  static Future<dynamic> _pedido(String metodo, String caminho, {Object?
corpo}) async {
    final uri = Uri.parse('$baseUrl$caminho');
    final cabecalhos = {
      'Content-Type': 'application/json',
      if (_token != null) 'Authorization': 'Bearer $_token',
    };
    final body = corpo == null ? null : jsonEncode(corpo);

    final http.Response resposta;
    try {
      final pedido = switch (metodo) {
        'GET' => http.get(uri, headers: cabecalhos),
        'POST' => http.post(uri, headers: cabecalhos, body: body),
        'PUT' => http.put(uri, headers: cabecalhos, body: body),
        _ => throw ArgumentError('Método desconhecido: $metodo'),
      };
      resposta = await pedido.timeout(const Duration(seconds: 10));
    } on TimeoutException {
```

```
      throw const ApiException('O servidor demorou demasiado a responder.');
    } on http.ClientException {
      throw const ApiException('Não foi possível ligar ao servidor. A API
está a correr?');
    }

    final dados = resposta.bodyBytes.isEmpty ? null :
jsonDecode(utf8.decode(resposta.bodyBytes));
    if (resposta.statusCode >= 400) {
      final mensagem = dados is Map && dados['erro'] is String ?
dados['erro'] as String : 'Erro ${resposta.statusCode}';
      throw ApiException(mensagem, resposta.statusCode);
    }
    return dados;
  }

  static List<T> _lista<T>(Object? dados, T Function(Map<String, dynamic>)
converter) =>
      [for (final item in dados as List) converter(item as Map<String,
dynamic>)];

  // ---------------------------------------------------------------
autenticação

  static Future<void> login(String email, String password) async {
    final dados = await _pedido('POST', '/auth/login', corpo: {'email':
email, 'password': password});
    _token = dados['token'] as String;
  }

  static Future<void> registo(String nome, String email, String password)
async {
    final dados = await _pedido('POST', '/auth/registo', corpo: {'nome':
nome, 'email': email, 'password': password});
    _token = dados['token'] as String;
  }

  // ---------------------------------------------------------------- perfil

  static Future<List<Modalidade>> modalidades() async =>
      _lista(await _pedido('GET', '/modalidades'), Modalidade.fromJson);

  static Future<Perfil> perfil() async =>
      Perfil.fromJson(await _pedido('GET', '/perfil') as Map<String,
dynamic>);
```

```
  static Future<void> atualizarPerfil(Map<String, Object?> campos) =>
_pedido('PUT', '/perfil', corpo: campos);

  static Future<void> atualizarModalidades(List<int> ids, {String nivel =
'Iniciante'}) => _pedido(
        'PUT',
        '/perfil/modalidades',
        corpo: {
          'modalidades': [for (final id in ids) {'modalidade_id': id,
'nivel': nivel}],
        },
      );

  // ---------------------------------------------------------------
jogadores

  static Future<List<Jogador>> jogadores({required int modalidadeId, int?
raioKm}) async {
    final raio = raioKm == null ? '' : '&raio=$raioKm';
    final dados = await _pedido('GET', '/jogadores?
modalidade=$modalidadeId$raio');
    return _lista(dados['jogadores'], Jogador.fromJson);
  }

  // ---------------------------------------------------------------- jogos

  static Future<List<Jogo>> jogos() async => _lista(await _pedido('GET',
'/jogos'), Jogo.fromJson);

  static Future<Jogo> jogo(int id) async => Jogo.fromJson(await
_pedido('GET', '/jogos/$id') as Map<String, dynamic>);

  static Future<int> criarJogo({
    required int modalidadeId,
    required DateTime dataHora,
    required String local,
    required String nivel,
    required String formato,
    required int capacidade,
    required int vagasAbertas,
    required double preco,
  }) async {
    final dados = await _pedido('POST', '/jogos', corpo: {
      'modalidade_id': modalidadeId,
      'data_hora': paraApi(dataHora),
      'local_nome': local,
```

```
      'nivel': nivel,
      'formato': formato,
      'capacidade': capacidade,
      'vagas_abertas': vagasAbertas,
      'preco': preco,
    });
    return dados['id'] as int;
  }

  /// Entra no jogo (ou, se já lá estiver, devolve o código que já tinha).
  static Future<String> entrarNoJogo(int id) async {
    final dados = await _pedido('POST', '/jogos/$id/entrar');
    return dados['codigo_checkin'] as String;
  }

  // ---------------------------------------------------------------
convites e avisos

  static Future<({List<Convite> enviados, List<Convite> recebidos})>
convites() async {
    final dados = await _pedido('GET', '/convites');
    return (
      enviados: _lista(dados['enviados'], Convite.fromJson),
      recebidos: _lista(dados['recebidos'], Convite.fromJson),
    );
  }

  static Future<void> enviarConvite({required int jogadorId, required int
modalidadeId, required DateTime dataHora}) =>
      _pedido('POST', '/convites', corpo: {
        'destinatario_id': jogadorId,
        'modalidade_id': modalidadeId,
        'data_hora': paraApi(dataHora),
      });

  static Future<void> responderConvite(int id, {required bool aceitar}) =>
      _pedido('POST', '/convites/$id/${aceitar ? 'aceitar' : 'recusar'}');

  static Future<List<Aviso>> avisos() async => _lista(await _pedido('GET',
'/avisos'), Aviso.fromJson);
}
```

12.5 `lib/data/models.dart`

Cada classe tem um construtor `fromJson`, que lê o mapa vindo da API. Os nomes em Dart seguem o estilo Dart (`distanciaKm`); os do JSON seguem a base de dados (`distancia_km`). Apaguem o ficheiro antigo `lib/data/mock_data.dart`.

```
/// Modelos com os dados que chegam da API (JSON → objetos Dart).
library;

double? _duplo(Object? valor) => valor == null ? null : (valor as
num).toDouble();

class Modalidade {
  const Modalidade({required this.id, required this.nome, required
this.descricao});

  factory Modalidade.fromJson(Map<String, dynamic> json) => Modalidade(
        id: json['id'] as int,
        nome: json['nome'] as String,
        descricao: json['descricao'] as String? ?? '',
      );

  final int id;
  final String nome;
  final String descricao;
}

class ModalidadeNivel {
  const ModalidadeNivel({required this.id, required this.nome, required
this.nivel});

  factory ModalidadeNivel.fromJson(Map<String, dynamic> json) =>
ModalidadeNivel(
        id: json['id'] as int,
        nome: json['nome'] as String,
        nivel: json['nivel'] as String,
      );

  final int id;
  final String nome;
  final String nivel;
}

class Disponibilidade {
  const Disponibilidade({required this.diaSemana, required this.horaInicio,
required this.horaFim});
```

```
  factory Disponibilidade.fromJson(Map<String, dynamic> json) =>
Disponibilidade(
        diaSemana: json['dia_semana'] as int,
        horaInicio: json['hora_inicio'] as String,
        horaFim: json['hora_fim'] as String,
      );

  /// 1 = segunda … 7 = domingo
  final int diaSemana;
  final String horaInicio;
  final String horaFim;
}

class Perfil {
  const Perfil({
    required this.id,
    required this.nome,
    required this.email,
    required this.localidade,
    required this.raioKm,
    required this.objetivo,
    required this.avaliacao,
    required this.modalidades,
    required this.disponibilidade,
  });

  factory Perfil.fromJson(Map<String, dynamic> json) => Perfil(
        id: json['id'] as int,
        nome: json['nome'] as String,
        email: json['email'] as String,
        localidade: json['localidade'] as String?,
        raioKm: json['raio_km'] as int,
        objetivo: json['objetivo'] as String,
        avaliacao: _duplo(json['avaliacao']),
        modalidades: [
          for (final m in json['modalidades'] as List)
ModalidadeNivel.fromJson(m as Map<String, dynamic>),
        ],
        disponibilidade: [
          for (final d in json['disponibilidade'] as List)
Disponibilidade.fromJson(d as Map<String, dynamic>),
        ],
      );

  final int id;
```

```
  final String nome;
  final String email;
  final String? localidade;
  final int raioKm;
  final String objetivo;
  final double? avaliacao;
  final List<ModalidadeNivel> modalidades;
  final List<Disponibilidade> disponibilidade;
}

class Criterio {
  const Criterio({required this.nome, required this.pontos, required
this.maximo});

  factory Criterio.fromJson(Map<String, dynamic> json) => Criterio(
        nome: json['nome'] as String,
        pontos: json['pontos'] as int,
        maximo: json['maximo'] as int,
      );

  final String nome;
  final int pontos;
  final int maximo;
}

class Jogador {
  const Jogador({
    required this.id,
    required this.nome,
    required this.modalidade,
    required this.nivel,
    required this.distanciaKm,
    required this.compatibilidade,
    required this.criterios,
    required this.avaliacao,
    required this.objetivo,
    required this.disponivelHoje,
  });

  factory Jogador.fromJson(Map<String, dynamic> json) => Jogador(
        id: json['id'] as int,
        nome: json['nome'] as String,
        modalidade: json['modalidade'] as String,
        nivel: json['nivel'] as String,
        distanciaKm: _duplo(json['distancia_km'])!,
        compatibilidade: json['compatibilidade'] as int,
```

```
        criterios: [
          for (final c in json['criterios'] as List) Criterio.fromJson(c as
Map<String, dynamic>),
        ],
        avaliacao: _duplo(json['avaliacao']),
        objetivo: json['objetivo'] as String,
        disponivelHoje: json['disponivel_hoje'] as String?,
      );

  final int id;
  final String nome;
  final String modalidade;
  final String nivel;
  final double distanciaKm;
  final int compatibilidade;
  final List<Criterio> criterios;
  final double? avaliacao;
  final String objetivo;

  /// Ex.: "19:00 – 21:00", ou null se hoje não está disponível.
  final String? disponivelHoje;

  String get inicial => nome.substring(0, 1);
}

class Participante {
  const Participante({required this.id, required this.nome, required
this.organizador, required this.nivel});

  factory Participante.fromJson(Map<String, dynamic> json) => Participante(
        id: json['id'] as int,
        nome: json['nome'] as String,
        organizador: json['organizador'] as bool,
        nivel: json['nivel'] as String?,
      );

  final int id;
  final String nome;
  final bool organizador;
  final String? nivel;
}

class Jogo {
  const Jogo({
    required this.id,
    required this.organizador,
```

```
    required this.modalidadeId,
    required this.modalidade,
    required this.dataHora,
    required this.localNome,
    required this.nivel,
    required this.formato,
    required this.capacidade,
    required this.ocupados,
    required this.vagas,
    required this.preco,
    required this.distanciaKm,
    this.participantes = const [],
    this.euParticipo = false,
  });

  factory Jogo.fromJson(Map<String, dynamic> json) => Jogo(
        id: json['id'] as int,
        organizador: json['organizador'] as String,
        modalidadeId: json['modalidade_id'] as int,
        modalidade: json['modalidade'] as String,
        dataHora: DateTime.parse(json['data_hora'] as String),
        localNome: json['local_nome'] as String,
        nivel: json['nivel'] as String,
        formato: json['formato'] as String,
        capacidade: json['capacidade'] as int,
        ocupados: json['ocupados'] as int,
        vagas: json['vagas'] as int,
        preco: _duplo(json['preco'])!,
        distanciaKm: _duplo(json['distancia_km']),
        participantes: [
          for (final p in json['participantes'] as List? ?? const [])
            Participante.fromJson(p as Map<String, dynamic>),
        ],
        euParticipo: json['eu_participo'] as bool? ?? false,
      );

  final int id;
  final String organizador;
  final int modalidadeId;
  final String modalidade;
  final DateTime dataHora;
  final String localNome;
  final String nivel;
  final String formato;
  final int capacidade;
  final int ocupados;
```

```
  final int vagas;
  final double preco;
  final double? distanciaKm;
  final List<Participante> participantes;
  final bool euParticipo;
}

class Convite {
  const Convite({
    required this.id,
    required this.estado,
    required this.dataHora,
    required this.modalidade,
    required this.jogadorId,
    required this.nome,
    required this.distanciaKm,
  });

  factory Convite.fromJson(Map<String, dynamic> json) => Convite(
        id: json['id'] as int,
        estado: json['estado'] as String,
        dataHora: DateTime.parse(json['data_hora'] as String),
        modalidade: json['modalidade'] as String,
        jogadorId: json['jogador_id'] as int,
        nome: json['nome'] as String,
        distanciaKm: _duplo(json['distancia_km']),
      );

  final int id;
  final String estado;
  final DateTime dataHora;
  final String modalidade;

  /// A outra pessoa: o destinatário (enviados) ou o remetente (recebidos).
  final int jogadorId;
  final String nome;
  final double? distanciaKm;
}

class Aviso {
  const Aviso({required this.id, required this.titulo, required this.detalhe,
required this.lida, required this.criadoEm});

  factory Aviso.fromJson(Map<String, dynamic> json) => Aviso(
        id: json['id'] as int,
        titulo: json['titulo'] as String,
```

```
        detalhe: json['detalhe'] as String? ?? '',
        lida: json['lida'] as bool,
        criadoEm: DateTime.parse(json['criado_em'] as String),
      );

  final int id;
  final String titulo;
  final String detalhe;
  final bool lida;
  final DateTime criadoEm;
}
```

12.6 `lib/utils/formatos.dart`

Todas as datas, horas, distâncias e preços são escritos aqui, para ficarem iguais ao mockup em todos os ecrãs. O `paraApi` converte uma data no formato que a API espera.

```
/// Datas, horas, distâncias e preços escritos como no mockup (pt-PT).
library;

const _diasCurtos = ['Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb', 'Dom'];
const _diasLongos = ['Segunda', 'Terça', 'Quarta', 'Quinta', 'Sexta',
'Sábado', 'Domingo'];
const _meses = [
  'janeiro', 'fevereiro', 'março', 'abril', 'maio', 'junho',
  'julho', 'agosto', 'setembro', 'outubro', 'novembro', 'dezembro',
];
const _mesesCurtos = ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun', 'Jul', 'Ago',
'Set', 'Out', 'Nov', 'Dez'];

String _doisDigitos(int n) => n.toString().padLeft(2, '0');

/// 10:00
String hora(DateTime d) =>
'${_doisDigitos(d.hour)}:${_doisDigitos(d.minute)}';

/// Segunda (dia_semana da BD: 1 = segunda)
String nomeDoDia(int diaSemana) => _diasLongos[diaSemana - 1];

/// Sáb, 3 de outubro
String dataCampo(DateTime d) => '${_diasCurtos[d.weekday - 1]}, ${d.day} de
${_meses[d.month - 1]}';

/// Sáb, 3 de outubro · 10:00
String dataMedia(DateTime d) => '${dataCampo(d)} · ${hora(d)}';
```

```
/// Sábado, 3 de outubro às 10:00
String dataLonga(DateTime d) => '${_diasLongos[d.weekday - 1]}, ${d.day} de
${_meses[d.month - 1]} às ${hora(d)}';

/// Sáb, 3 Out · 10:00
String dataCurta(DateTime d) => '${_diasCurtos[d.weekday - 1]}, ${d.day}
${_mesesCurtos[d.month - 1]} · ${hora(d)}';

/// hoje às 19:00 · amanhã às 10:00 · quarta às 18:00
String diaHora(DateTime d) {
  final hoje = DateTime.now();
  final dia = DateTime(d.year, d.month, d.day);
  final diferenca = dia.difference(DateTime(hoje.year, hoje.month,
hoje.day)).inDays;
  final quando = switch (diferenca) {
    0 => 'hoje',
    1 => 'amanhã',
    _ => _diasLongos[d.weekday - 1].toLowerCase(),
  };
  return '$quando às ${hora(d)}';
}

/// Há 5 minutos · Há 2 horas · Ontem · Há 3 dias
String tempoRelativo(DateTime d) {
  final passou = DateTime.now().difference(d);
  if (passou.inMinutes < 1) return 'Agora';
  if (passou.inMinutes < 60) return 'Há ${passou.inMinutes} minutos';
  if (passou.inHours < 24) return 'Há ${passou.inHours} horas';
  if (passou.inDays < 2) return 'Ontem';
  return 'Há ${passou.inDays} dias';
}

/// 1,8 km
String km(double valor) => '${valor.toStringAsFixed(1).replaceAll('.', ',')}
km';

/// 8 € · 7,50 €
String euros(double valor) => valor == valor.roundToDouble()
    ? '${valor.toInt()} €'
    : '${valor.toStringAsFixed(2).replaceAll('.', ',')} €';

/// "18:00" → "18h" · "13:30" → "13h30"
String horaCurta(String hhmm) {
  final partes = hhmm.split(':');
  return partes[1] == '00' ? '${int.parse(partes[0])}h' :
'${int.parse(partes[0])}h${partes[1]}';
```

```
}

/// Formato que a API espera: 2026-10-31 10:00:00
String paraApi(DateTime d) =>
    '${d.year}-${_doisDigitos(d.month)}-${_doisDigitos(d.day)}
${hora(d)}:00';

/// Hora a propor num convite: o início da disponibilidade de hoje, ou daqui
a 1 hora.
DateTime inicioConvite(String? disponivelHoje) {
  final agora = DateTime.now();
  if (disponivelHoje != null) {
    final partes = disponivelHoje.substring(0, 5).split(':');
    final inicio = DateTime(agora.year, agora.month, agora.day,
int.parse(partes[0]), int.parse(partes[1]));
    if (inicio.isAfter(agora)) return inicio;
  }
  return DateTime(agora.year, agora.month, agora.day, agora.hour + 1);
}
```

12.7 `lib/widgets/async_view.dart`

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import 'buttons.dart';

/// Mostra um indicador enquanto os dados chegam da API, uma mensagem se
/// falhar (com "Tentar outra vez") e o conteúdo quando estiverem prontos.
class AsyncView<T> extends StatelessWidget {
  const AsyncView({super.key, required this.future, required this.builder,
this.onRetry});

  final Future<T> future;
  final Widget Function(BuildContext context, T data) builder;
  final VoidCallback? onRetry;

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<T>(
      future: future,
      builder: (context, snapshot) {
        if (snapshot.connectionState != ConnectionState.done) {
          return const Padding(
            padding: EdgeInsets.symmetric(vertical: 48),
```

```
            child: Center(child: CircularProgressIndicator(color:
AppColors.primary)),
          );
        }
        if (snapshot.hasError) {
          return Padding(
            padding: const EdgeInsets.symmetric(vertical: 48, horizontal:
20),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                Text(
                  snapshot.error.toString(),
                  textAlign: TextAlign.center,
                  style: AppText.body(15, color: AppColors.muted),
                ),
                if (onRetry != null) ...[
                  const SizedBox(height: 16),
                  SizedBox(
                    width: 200,
                    child: AppButton.small(label: 'Tentar outra vez',
outlined: true, onPressed: onRetry),
                  ),
                ],
              ],
            ),
          );
        }
        return builder(context, snapshot.data as T);
      },
    );
  }
}

/// Texto cinzento para listas vazias.
class EmptyText extends StatelessWidget {
  const EmptyText(this.text, {super.key});

  final String text;

  @override
  Widget build(BuildContext context) => Padding(
        padding: const EdgeInsets.symmetric(vertical: 8),
        child: Text(text, style: AppText.body(15, color: AppColors.muted)),
      );
}
```

12.8 Ecrã 01 — `login_screen.dart`

Passa a `StatefulWidget`, com 2 `TextEditingController` para ler o que foi escrito. **Entrar** chama `Api.login` e, se correr bem, abre o `HomeShell`. **Criar conta** abre o ecrã novo de registo. Os campos vêm preenchidos com o Sebastião do `seed.sql` para testarem mais depressa.

```
import 'package:flutter/material.dart';

import '../services/api.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import '../widgets/fields.dart';
import '../widgets/logo.dart';
import 'home_shell.dart';
import 'register_screen.dart';

/// 01 Login
class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  // Preenchido com o utilizador de exemplo do seed.sql para testar mais
depressa.
  final _email = TextEditingController(text: 'sebastiao@email.com');
  final _password = TextEditingController(text: '12345678');
  bool _aEntrar = false;

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  Future<void> _entrar() async {
    setState(() => _aEntrar = true);
    try {
      await Api.login(_email.text.trim(), _password.text);
      if (!mounted) return;
```

```
      Navigator.of(context).pushReplacement(MaterialPageRoute(builder: (_) =>
const HomeShell()));
    } on ApiException catch (erro) {
      if (mounted) showInfo(context, erro.message);
    } finally {
      if (mounted) setState(() => _aEntrar = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.lightStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Container(
                color: AppColors.forest,
                padding: EdgeInsets.fromLTRB(28, topInset(context, 88), 28,
44),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    const WhoPlayLogo(),
                    const SizedBox(height: 14),
                    ConstrainedBox(
                      constraints: const BoxConstraints(maxWidth: 250),
                      child: Text(
                        'Encontra pessoas. Marca o jogo. Vai jogar.',
                        style: AppText.body(19, color: AppColors.white,
height: 1.4),
                      ),
                    ),
                  ],
                ),
              ),
              Expanded(
                child: Padding(
                  padding: EdgeInsets.fromLTRB(28, 32, 28, 28 +
bottomInset(context)),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.stretch,
                    children: [
                      Text('Entrar', style: AppText.display(28,
```

```
letterSpacing: -0.5)),
                      const SizedBox(height: 18),
                      AppTextField(
                        label: 'Email',
                        controller: _email,
                        keyboardType: TextInputType.emailAddress,
                      ),
                      const SizedBox(height: 18),
                      AppTextField(label: 'Palavra-passe', controller:
_password, obscureText: true),
                      const SizedBox(height: 18),
                      Align(
                        alignment: Alignment.centerRight,
                        child: InkWell(
                          onTap: () => showInfo(context, 'Recuperação de
palavra-passe em breve.'),
                          child: Padding(
                            padding: const EdgeInsets.symmetric(vertical: 6),
                            child: Text(
                              'Esqueci-me da palavra-passe',
                              style: AppText.body(14, weight:
FontWeight.w600, color: AppColors.primary),
                            ),
                          ),
                        ),
                      ),
                      const SizedBox(height: 18),
                      AppButton(label: _aEntrar ? 'A entrar…' : 'Entrar',
onPressed: _aEntrar ? null : _entrar),
                      const SizedBox(height: 18),
                      const Spacer(),
                      Text(
                        'Ainda não tens conta?',
                        textAlign: TextAlign.center,
                        style: AppText.body(14, color: AppColors.muted),
                      ),
                      const SizedBox(height: 12),
                      AppButton(
                        label: 'Criar conta',
                        outlined: true,
                        onPressed: () => Navigator.of(context).push(
                          MaterialPageRoute(builder: (_) => const
RegisterScreen()),
                        ),
                      ),
                    ],
```

```
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

12.9 Ecrã novo — `register_screen.dart`

O mockup não tinha um ecrã para escrever o nome, o email e a palavra-passe, e sem eles não há conta (RF01). Este ecrã segue o estilo do "Criar jogo". Depois do registo, abre o Onboarding.

```
import 'package:flutter/material.dart';

import '../services/api.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import '../widgets/fields.dart';
import 'onboarding_screen.dart';

/// Criar conta (RF01). Não está no mockup: segue o estilo do ecrã "Criar
jogo".
class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _nome = TextEditingController();
  final _email = TextEditingController();
  final _password = TextEditingController();
  bool _aGuardar = false;

  @override
  void dispose() {
    _nome.dispose();
    _email.dispose();
```

```
    _password.dispose();
    super.dispose();
  }

  Future<void> _criarConta() async {
    setState(() => _aGuardar = true);
    try {
      await Api.registo(_nome.text.trim(), _email.text.trim(),
_password.text);
      if (!mounted) return;
      Navigator.of(context).pushReplacement(MaterialPageRoute(builder: (_) =>
const OnboardingScreen()));
    } on ApiException catch (erro) {
      if (mounted) showInfo(context, erro.message);
    } finally {
      if (mounted) setState(() => _aGuardar = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          padding: EdgeInsets.fromLTRB(20, topInset(context, 48), 20, 28 +
bottomInset(context)),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Row(
                children: [
                  CircleIconButton(
                    semanticLabel: 'Voltar',
                    onPressed: () => Navigator.of(context).pop(),
                    child: const SvgIcon(AppIcon.back),
                  ),
                  const SizedBox(width: 12),
                  const ScreenTitle('Criar conta'),
                ],
              ),
              const SizedBox(height: 24),
              AppTextField(label: 'Nome', controller: _nome),
              const SizedBox(height: 18),
              AppTextField(label: 'Email', controller: _email, keyboardType:
TextInputType.emailAddress),
```

```
              const SizedBox(height: 18),
              AppTextField(label: 'Palavra-passe (mínimo 8 caracteres)',
controller: _password, obscureText: true),
              const SizedBox(height: 24),
              const Spacer(),
              AppButton(
                label: _aGuardar ? 'A criar…' : 'Criar conta',
                onPressed: _aGuardar ? null : _criarConta,
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

12.10 Ecrã 02 — `onboarding_screen.dart`

Os desportos passam a vir de `GET /modalidades`, e os escolhidos são guardados por id. **Continuar** grava-os com `PUT /perfil/modalidades` (nível Iniciante, por agora). Grava também uma localização provisória (o centro de Lisboa), porque sem ela a procura de jogadores não funciona; o passo seguinte do projeto é usar o GPS com o pacote `geolocator`.

```
import 'package:flutter/material.dart';

import '../data/models.dart';
import '../services/api.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../widgets/app_icons.dart';
import '../widgets/async_view.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import 'home_shell.dart';

/// 02 Onboarding — desportos (passo 1 de 3)
class OnboardingScreen extends StatefulWidget {
  const OnboardingScreen({super.key});

  @override
  State<OnboardingScreen> createState() => _OnboardingScreenState();
}
```

```
class _OnboardingScreenState extends State<OnboardingScreen> {
  late Future<List<Modalidade>> _modalidades = Api.modalidades();
  final _selected = <int>{};
  bool _aGuardar = false;

  void _toggle(int id) => setState(() {
        if (!_selected.remove(id)) _selected.add(id);
      });

  Future<void> _continuar() async {
    setState(() => _aGuardar = true);
    try {
      await Api.atualizarModalidades(_selected.toList());
      // Localização provisória (centro de Lisboa) até haver GPS (pacote
geolocator).
      await Api.atualizarPerfil({'localidade': 'Lisboa', 'latitude': 38.7223,
'longitude': -9.1393});
      if (!mounted) return;
      Navigator.of(context).pushAndRemoveUntil(
        MaterialPageRoute(builder: (_) => const HomeShell(initialTab:
HomeShell.profileTab)),
        (_) => false,
      );
    } on ApiException catch (erro) {
      if (mounted) showInfo(context, erro.message);
    } finally {
      if (mounted) setState(() => _aGuardar = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final count = _selected.length;
    final countLabel = count == 1 ? '1 desporto escolhido' : '$count
desportos escolhidos';

    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          padding: EdgeInsets.fromLTRB(24, topInset(context, 56), 24, 28 +
bottomInset(context)),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const _StepProgress(step: 1, total: 3),
```

```
              const SizedBox(height: 28),
              Text(
                'Que desportos queres jogar?',
                style: AppText.display(30, letterSpacing: -0.6, height:
1.12),
              ),
              const SizedBox(height: 8),
              Text(
                'Escolhe um ou mais. Podes mudar isto no teu perfil.',
                style: AppText.body(16, color: AppColors.muted, height:
1.45),
              ),
              const SizedBox(height: 24),
              AsyncView<List<Modalidade>>(
                future: _modalidades,
                onRetry: () => setState(() => _modalidades =
Api.modalidades()),
                builder: (context, modalidades) => Column(
                  children: [
                    for (var i = 0; i < modalidades.length; i += 2) ...[
                      if (i > 0) const SizedBox(height: 12),
                      Row(
                        children: [
                          Expanded(child: _tile(modalidades[i])),
                          const SizedBox(width: 12),
                          Expanded(
                            child: i + 1 < modalidades.length ?
_tile(modalidades[i + 1]) : const SizedBox(),
                          ),
                        ],
                      ),
                    ],
                  ],
                ),
              ),
              const SizedBox(height: 24),
              const Spacer(),
              Text(
                '$countLabel · a seguir: nível e disponibilidade',
                textAlign: TextAlign.center,
                style: AppText.body(14, color: AppColors.muted),
              ),
              const SizedBox(height: 10),
              AppButton(
                label: _aGuardar ? 'A guardar…' : 'Continuar',
                onPressed: count == 0 || _aGuardar ? null : _continuar,
```

```
              ),
            ],
          ),
        ),
      ),
    );
  }

  Widget _tile(Modalidade modalidade) => _SportTile(
        modalidade: modalidade,
        selected: _selected.contains(modalidade.id),
        onTap: () => _toggle(modalidade.id),
      );
}
```

As classes `_StepProgress` e `_SportTile` ficam como no guia anterior, com 2 mudanças no `_SportTile`: o campo `final Sport sport;` passa a `final Modalidade modalidade;` (e o construtor recebe `required this.modalidade`), e `sport.name` / `sport.subtitle` passam a `modalidade.nome` / `modalidade.descricao`.

12.11 Ecrã 03 — `profile_screen.dart`

Carrega `GET /perfil`. Puxar o ecrã para baixo recarrega os dados (`RefreshIndicator`). No fundo há um botão novo, **Terminar sessão** (RF03), que esquece o token e volta ao Login. Os desportos com nível Intermédio ou Avançado aparecem com a etiqueta lima.

```
import 'package:flutter/material.dart';

import '../data/models.dart';
import '../services/api.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../utils/formatos.dart';
import '../widgets/app_icons.dart';
import '../widgets/async_view.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import 'login_screen.dart';

/// 03 Perfil
class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
```

```
}

class _ProfileScreenState extends State<ProfileScreen> {
  late Future<Perfil> _perfil = Api.perfil();

  void _recarregar() => setState(() => _perfil = Api.perfil());

  void _terminarSessao() {
    Api.terminarSessao();
    Navigator.of(context).pushAndRemoveUntil(
      MaterialPageRoute(builder: (_) => const LoginScreen()),
      (_) => false,
    );
  }

  @override
  Widget build(BuildContext context) {
    return RefreshIndicator(
      color: AppColors.primary,
      onRefresh: () async {
        _recarregar();
        await _perfil;
      },
      child: SingleChildScrollView(
        physics: const AlwaysScrollableScrollPhysics(),
        padding: EdgeInsets.fromLTRB(20, topInset(context, 52), 20, 20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                const ScreenTitle('Perfil'),
                AppButton.pill(
                  label: 'Editar',
                  onPressed: () => showInfo(context, 'Edição do perfil em
breve.'),
                ),
              ],
            ),
            const SizedBox(height: 18),
            AsyncView<Perfil>(
              future: _perfil,
              onRetry: _recarregar,
              builder: (context, perfil) => _ProfileBody(perfil: perfil,
onLogout: _terminarSessao),
```

```
            ),
          ],
        ),
      ),
    );
  }
}

class _ProfileBody extends StatelessWidget {
  const _ProfileBody({required this.perfil, required this.onLogout});

  final Perfil perfil;
  final VoidCallback onLogout;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Container(
          padding: const EdgeInsets.all(18),
          decoration: BoxDecoration(color: AppColors.forest, borderRadius:
BorderRadius.circular(20)),
          child: Row(
            children: [
              Avatar(
                perfil.nome.substring(0, 1),
                size: 64,
                fontSize: 28,
                background: AppColors.lime,
                foreground: AppColors.ink,
              ),
              const SizedBox(width: 16),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(perfil.nome, style: AppText.display(22, color:
AppColors.white)),
                    const SizedBox(height: 4),
                    Row(
                      children: [
                        const SvgIcon(AppIcon.pin, size: 16, color:
AppColors.white),
                        const SizedBox(width: 6),
                        Flexible(
```

```
                          child: Text(
                            '${perfil.localidade ?? 'Sem localização'} · raio
até ${perfil.raioKm} km',
                            style: AppText.body(14, color: AppColors.white),
                          ),
                        ),
                      ],
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
        const SizedBox(height: 18),
        Section(
          title: 'Desportos e nível',
          child: perfil.modalidades.isEmpty
              ? const EmptyText('Ainda não escolheste desportos.')
              : ListCard(
                  children: [
                    for (final m in perfil.modalidades)
                      _SportLevelRow(m.nome, m.nivel, highlight: m.nivel !=
'Iniciante'),
                  ],
                ),
        ),
        const SizedBox(height: 18),
        Section(
          title: 'Disponibilidade',
          child: perfil.disponibilidade.isEmpty
              ? const EmptyText('Ainda não indicaste horários.')
              : ListCard(
                  children: [
                    for (final d in perfil.disponibilidade)
                      _AvailabilityRow(nomeDoDia(d.diaSemana),
'${horaCurta(d.horaInicio)} – ${horaCurta(d.horaFim)}'),
                  ],
                ),
        ),
        const SizedBox(height: 18),
        Section(
          title: 'Objetivo',
          child: SurfaceCard(
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical:
14),
```

```
            child: Text(perfil.objetivo, style: AppText.body(16, weight:
FontWeight.w600)),
          ),
        ),
        const SizedBox(height: 24),
        AppButton.small(label: 'Terminar sessão', outlined: true, onPressed:
onLogout),
      ],
    );
  }
}
```

As classes `_SportLevelRow` e `_AvailabilityRow` ficam iguais ao guia anterior.

12.12 Ecrã 04 — `find_players_screen.dart`

Os filtros passam a mudar o pedido à API: **Padel** e **Ténis** mudam a `modalidade` (ids 1 e 2), e **Até 5 km** envia `raio=5`. **Hoje** filtra na própria app quem tem `disponivelHoje`. **Convidar** cria um convite para o início da disponibilidade de hoje desse jogador. A percentagem e a disponibilidade ficam verdes quando a compatibilidade é de 80% ou mais.

```
import 'package:flutter/material.dart';

import '../data/models.dart';
import '../services/api.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../utils/formatos.dart';
import '../widgets/app_icons.dart';
import '../widgets/async_view.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import 'player_profile_screen.dart';

/// 04 Encontrar jogadores
class FindPlayersScreen extends StatefulWidget {
  const FindPlayersScreen({super.key, this.onInviteSent});

  final VoidCallback? onInviteSent;

  @override
  State<FindPlayersScreen> createState() => _FindPlayersScreenState();
}

class _FindPlayersScreenState extends State<FindPlayersScreen> {
  static const _filtrosDesporto = [(1, 'Padel'), (2, 'Ténis')]; // ids da
```

```
tabela modalidades

  int _modalidadeId = 1;
  bool _within5km = false;
  bool _today = false;
  late Future<List<Jogador>> _jogadores = _carregar();

  Future<List<Jogador>> _carregar() =>
      Api.jogadores(modalidadeId: _modalidadeId, raioKm: _within5km ? 5 :
null);

  void _recarregar() => setState(() => _jogadores = _carregar());

  Future<void> _openProfile(Jogador jogador) async {
    final invited = await Navigator.of(context).push<bool>(
      MaterialPageRoute(builder: (_) => PlayerProfileScreen(jogador: jogador,
modalidadeId: _modalidadeId)),
    );
    if (invited == true) widget.onInviteSent?.call();
  }

  Future<void> _convidar(Jogador jogador) async {
    try {
      await Api.enviarConvite(
        jogadorId: jogador.id,
        modalidadeId: _modalidadeId,
        dataHora: inicioConvite(jogador.disponivelHoje),
      );
      if (mounted) showInfo(context, 'Convite enviado a ${jogador.nome}.');
    } on ApiException catch (erro) {
      if (mounted) showInfo(context, erro.message);
    }
  }

  @override
  Widget build(BuildContext context) {
    return RefreshIndicator(
      color: AppColors.primary,
      onRefresh: () async {
        _recarregar();
        await _jogadores;
      },
      child: SingleChildScrollView(
        physics: const AlwaysScrollableScrollPhysics(),
        padding: EdgeInsets.fromLTRB(20, topInset(context, 52), 20, 20),
        child: Column(
```

```
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const ScreenTitle('Encontrar jogadores'),
            const SizedBox(height: 14),
            SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              clipBehavior: Clip.none,
              child: Row(
                children: [
                  for (final (id, nome) in _filtrosDesporto) ...[
                    PillChip(
                      label: nome,
                      selected: _modalidadeId == id,
                      onTap: () {
                        _modalidadeId = id;
                        _recarregar();
                      },
                    ),
                    const SizedBox(width: 8),
                  ],
                  PillChip(
                    label: 'Até 5 km',
                    selected: _within5km,
                    onTap: () {
                      _within5km = !_within5km;
                      _recarregar();
                    },
                  ),
                  const SizedBox(width: 8),
                  PillChip(label: 'Hoje', selected: _today, onTap: () =>
setState(() => _today = !_today)),
                ],
              ),
            ),
            const SizedBox(height: 14),
            AsyncView<List<Jogador>>(
              future: _jogadores,
              onRetry: _recarregar,
              builder: (context, todos) {
                final jogadores = _today ? todos.where((j) =>
j.disponivelHoje != null).toList() : todos;
                return Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    Text(
                      jogadores.length == 1
```

```
                          ? '1 jogador compatível perto de ti'
                          : '${jogadores.length} jogadores compatíveis perto
de ti',
                      style: AppText.body(14, color: AppColors.muted),
                    ),
                    const SizedBox(height: 14),
                    if (jogadores.isEmpty) const EmptyText('Ainda não há
jogadores desta modalidade perto de ti.'),
                    for (var i = 0; i < jogadores.length; i++) ...[
                      if (i > 0) const SizedBox(height: 12),
                      PlayerCard(
                        jogador: jogadores[i],
                        onViewProfile: () => _openProfile(jogadores[i]),
                        onInvite: () => _convidar(jogadores[i]),
                      ),
                    ],
                  ],
                );
              },
            ),
          ],
        ),
      ),
    );
  }
}

class PlayerCard extends StatelessWidget {
  const PlayerCard({super.key, required this.jogador, this.onViewProfile,
this.onInvite});

  final Jogador jogador;
  final VoidCallback? onViewProfile;
  final VoidCallback? onInvite;

  @override
  Widget build(BuildContext context) {
    final destaque = jogador.compatibilidade >= 80;
    final hoje = jogador.disponivelHoje;
    final availabilityColor = destaque && hoje != null ? AppColors.primary :
AppColors.muted;
    return SurfaceCard(
      radius: 18,
      padding: const EdgeInsets.all(14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
```

```
        children: [
          Row(
            children: [
              Avatar(jogador.inicial),
              const SizedBox(width: 12),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(jogador.nome, style: AppText.body(17, weight:
FontWeight.w700)),
                    const SizedBox(height: 2),
                    Text(
                      '${jogador.modalidade} · ${jogador.nivel} ·
${km(jogador.distanciaKm)}',
                      style: AppText.body(14, color: AppColors.muted),
                    ),
                  ],
                ),
              ),
              const SizedBox(width: 12),
              Tag(
                '${jogador.compatibilidade}%',
                display: true,
                fontSize: 18,
                background: destaque ? AppColors.lime : AppColors.chip,
              ),
            ],
          ),
          const SizedBox(height: 12),
          Row(
            children: [
              SvgIcon(AppIcon.clock, size: 16, color: availabilityColor),
              const SizedBox(width: 6),
              Text(
                hoje != null ? 'Disponível hoje $hoje' : 'Sem disponibilidade
hoje',
                style: AppText.body(14, weight: FontWeight.w600, color:
availabilityColor),
              ),
            ],
          ),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(child: AppButton.small(label: 'Ver perfil', outlined:
```

```
true, onPressed: onViewProfile)),
              const SizedBox(width: 8),
              Expanded(child: AppButton.small(label: 'Convidar', onPressed:
onInvite)),
            ],
          ),
        ],
      ),
    );
  }
}
```

12.13 Ecrã 05 — `player_profile_screen.dart`

Não precisa de pedir nada à API para se mostrar: recebe o `Jogador` já carregado na lista, que traz os 5 critérios. **Enviar convite** chama a API e fecha o ecrã com `pop(true)`, e o `HomeShell` salta para os Convites.

```
import 'package:flutter/material.dart';

import '../data/models.dart';
import '../services/api.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../utils/formatos.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';

/// 05 Perfil do jogador. Devolve `true` ao fazer pop se o convite foi
enviado.
class PlayerProfileScreen extends StatefulWidget {
  const PlayerProfileScreen({super.key, required this.jogador, required
this.modalidadeId});

  final Jogador jogador;
  final int modalidadeId;

  @override
  State<PlayerProfileScreen> createState() => _PlayerProfileScreenState();
}

class _PlayerProfileScreenState extends State<PlayerProfileScreen> {
  bool _aEnviar = false;

  Future<void> _enviarConvite() async {
```

```
    setState(() => _aEnviar = true);
    try {
      await Api.enviarConvite(
        jogadorId: widget.jogador.id,
        modalidadeId: widget.modalidadeId,
        dataHora: inicioConvite(widget.jogador.disponivelHoje),
      );
      if (mounted) Navigator.of(context).pop(true);
    } on ApiException catch (erro) {
      if (mounted) showInfo(context, erro.message);
    } finally {
      if (mounted) setState(() => _aEnviar = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final jogador = widget.jogador;
    final avaliacao = jogador.avaliacao;
    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          padding: EdgeInsets.fromLTRB(20, topInset(context, 48), 20, 28 +
bottomInset(context)),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Align(
                alignment: Alignment.centerLeft,
                child: CircleIconButton(
                  semanticLabel: 'Voltar',
                  onPressed: () => Navigator.of(context).pop(),
                  child: const SvgIcon(AppIcon.back),
                ),
              ),
              const SizedBox(height: 16),
              Row(
                children: [
                  Avatar(jogador.inicial, size: 72, fontSize: 30),
                  const SizedBox(width: 16),
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(jogador.nome, style: AppText.display(28,
```

```
letterSpacing: -0.5)),
                        const SizedBox(height: 4),
                        Text(
                          '${jogador.modalidade} · ${jogador.nivel} ·
${km(jogador.distanciaKm)}',
                          style: AppText.body(15, color: AppColors.muted),
                        ),
                        const SizedBox(height: 4),
                        Row(
                          children: [
                            const SvgIcon(AppIcon.star, size: 16,
strokeWidth: 1.5),
                            const SizedBox(width: 5),
                            Text(
                              avaliacao == null
                                  ? 'Sem avaliações'
                                  : '${avaliacao.toString().replaceAll('.',
',')} de avaliação',
                              style: AppText.body(15, weight:
FontWeight.w700),
                            ),
                          ],
                        ),
                      ],
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 16),
              _CompatibilityCard(jogador: jogador),
              const SizedBox(height: 16),
              ListCard(
                children: [
                  _InfoRow('Disponível hoje', jogador.disponivelHoje ?? '—'),
                  _InfoRow('Objetivo', jogador.objetivo),
                ],
              ),
              const SizedBox(height: 16),
              const Spacer(),
              AppButton(
                label: _aEnviar ? 'A enviar…' : 'Enviar convite',
                onPressed: _aEnviar ? null : _enviarConvite,
              ),
            ],
          ),
        ),
```

```
      ),
    );
  }
}

class _CompatibilityCard extends StatelessWidget {
  const _CompatibilityCard({required this.jogador});

  final Jogador jogador;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(color: AppColors.forest, borderRadius:
BorderRadius.circular(20)),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            crossAxisAlignment: CrossAxisAlignment.baseline,
            textBaseline: TextBaseline.alphabetic,
            children: [
              Text(
                'Compatibilidade contigo',
                style: AppText.body(15, weight: FontWeight.w600, color:
AppColors.white),
              ),
              Text(
                '${jogador.compatibilidade}%',
                style: AppText.display(44, weight: FontWeight.w800, color:
AppColors.lime, height: 1),
              ),
            ],
          ),
          for (final criterio in jogador.criterios) ...[
            const SizedBox(height: 14),
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Text(criterio.nome, style: AppText.body(14, color:
AppColors.white)),
                Text(
                  '${criterio.pontos} / ${criterio.maximo}',
                  style: AppText.body(14, weight: FontWeight.w700, color:
```

```
AppColors.white),
                ),
              ],
            ),
            const SizedBox(height: 6),
            ThinBar(
              value: criterio.pontos / criterio.maximo,
              color: AppColors.lime,
              track: AppColors.forestLight,
            ),
          ],
        ],
      ),
    );
  }
}
```

A classe `_InfoRow` fica igual ao guia anterior.

12.14 Ecrã 06 — `invites_screen.dart`

Carrega os convites e os avisos (2 pedidos) e junta-os num *record* (`_DadosConvites`). Cada convite passa a ter o seu cartão (`_SentCard` , `_ReceivedCard`). Depois de Aceitar ou Recusar, o ecrã volta a pedir os dados à API. Nos avisos sem `detalhe` , o subtítulo mostra há quanto tempo chegaram ("Há 5 minutos").

```
import 'package:flutter/material.dart';

import '../data/models.dart';
import '../services/api.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../utils/formatos.dart';
import '../widgets/async_view.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';

typedef _DadosConvites = ({List<Convite> enviados, List<Convite> recebidos,
List<Aviso> avisos});

/// 06 Convites e avisos
class InvitesScreen extends StatefulWidget {
  const InvitesScreen({super.key});

  @override
  State<InvitesScreen> createState() => _InvitesScreenState();
```

```
}

class _InvitesScreenState extends State<InvitesScreen> {
  late Future<_DadosConvites> _dados = _carregar();

  Future<_DadosConvites> _carregar() async {
    final convites = await Api.convites();
    final avisos = await Api.avisos();
    return (enviados: convites.enviados, recebidos: convites.recebidos,
avisos: avisos);
  }

  void _recarregar() => setState(() => _dados = _carregar());

  Future<void> _answer(Convite convite, bool accepted) async {
    try {
      await Api.responderConvite(convite.id, aceitar: accepted);
      if (!mounted) return;
      showInfo(context, accepted ? 'Convite aceite.' : 'Convite recusado.');
      _recarregar();
    } on ApiException catch (erro) {
      if (mounted) showInfo(context, erro.message);
    }
  }

  @override
  Widget build(BuildContext context) {
    return RefreshIndicator(
      color: AppColors.primary,
      onRefresh: () async {
        _recarregar();
        await _dados;
      },
      child: SingleChildScrollView(
        physics: const AlwaysScrollableScrollPhysics(),
        padding: EdgeInsets.fromLTRB(20, topInset(context, 52), 20, 20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const ScreenTitle('Convites'),
            const SizedBox(height: 16),
            AsyncView<_DadosConvites>(
              future: _dados,
              onRetry: _recarregar,
              builder: (context, dados) => Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
```

```
                children: [
                  Section(
                    title: 'Enviados',
                    child: dados.enviados.isEmpty
                        ? const EmptyText('Nenhum convite à espera de
resposta.')
                        : Column(
                            children: [
                              for (final c in dados.enviados) ...[
                                if (c != dados.enviados.first) const
SizedBox(height: 10),
                                _SentCard(c),
                              ],
                            ],
                          ),
                  ),
                  const SizedBox(height: 16),
                  Section(
                    title: 'Recebidos',
                    child: dados.recebidos.isEmpty
                        ? const EmptyText('Não tens convites por responder.')
                        : Column(
                            children: [
                              for (final c in dados.recebidos) ...[
                                if (c != dados.recebidos.first) const
SizedBox(height: 10),
                                _ReceivedCard(
                                  c,
                                  onDecline: () => _answer(c, false),
                                  onAccept: () => _answer(c, true),
                                ),
                              ],
                            ],
                          ),
                  ),
                  const SizedBox(height: 16),
                  Section(
                    title: 'Avisos',
                    child: dados.avisos.isEmpty
                        ? const EmptyText('Sem avisos.')
                        : ListCard(
                            radius: 18,
                            children: [
                              for (final a in dados.avisos)
                                _NoticeRow(
                                  a.titulo,
```

```
                                  a.detalhe.isNotEmpty ? a.detalhe :
tempoRelativo(a.criadoEm),
                                  unread: !a.lida,
                                ),
                              ],
                            ),
                        ),
                      ],
                    ),
                  ),
                ],
              ),
            ),
        );
  }
}

class _SentCard extends StatelessWidget {
  const _SentCard(this.convite);

  final Convite convite;

  @override
  Widget build(BuildContext context) {
    return SurfaceCard(
      radius: 18,
      padding: const EdgeInsets.all(14),
      child: Row(
        children: [
          Avatar(convite.nome.substring(0, 1)),
          const SizedBox(width: 12),
          Expanded(
            child: _TitleSubtitle(
              title: convite.nome,
              subtitle: '${convite.modalidade} ·
${diaHora(convite.dataHora)}',
            ),
          ),
          const SizedBox(width: 12),
          const Tag('A aguardar', fontSize: 13),
        ],
      ),
    );
  }
}
```

```
class _ReceivedCard extends StatelessWidget {
  const _ReceivedCard(this.convite, {required this.onDecline, required
this.onAccept});

  final Convite convite;
  final VoidCallback onDecline;
  final VoidCallback onAccept;

  @override
  Widget build(BuildContext context) {
    final distancia = convite.distanciaKm;
    return SurfaceCard(
      radius: 18,
      padding: const EdgeInsets.all(14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Row(
            children: [
              Avatar(convite.nome.substring(0, 1)),
              const SizedBox(width: 12),
              Expanded(
                child: _TitleSubtitle(
                  title: '${convite.nome} convidou-te para jogar',
                  subtitle: [
                    convite.modalidade,
                    diaHora(convite.dataHora),
                    if (distancia != null) km(distancia),
                  ].join(' · '),
                ),
              ),
            ],
          ),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(child: AppButton.small(label: 'Recusar', outlined:
true, onPressed: onDecline)),
              const SizedBox(width: 8),
              Expanded(child: AppButton.small(label: 'Aceitar', onPressed:
onAccept)),
            ],
          ),
        ],
```

```
      ),
    );
  }
}
```

As classes `_TitleSubtitle` e `_NoticeRow` ficam iguais ao guia anterior.

12.15 Ecrã 07 — `create_game_screen.dart`

O desporto vem de `GET /modalidades`, a data começa em "amanhã" e o calendário já não deixa escolher dias passados. **Criar jogo** envia tudo para `POST /jogos` e abre o detalhe do jogo criado. O preço "8 €" é convertido para o número 8. Este bloco substitui tudo, do início do ficheiro até ao fim da classe `_CreateGameScreenState`; as classes `_ChoiceGroup` e `_StepperButton` ficam iguais.

```
import 'package:flutter/material.dart';

import '../data/models.dart';
import '../services/api.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../utils/formatos.dart';
import '../widgets/app_icons.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import '../widgets/fields.dart';
import 'game_detail_screen.dart';

const _levels = ['Iniciante', 'Intermédio', 'Avançado'];
const _formats = ['Amigável', 'Competitivo'];

/// 07 Criar jogo
class CreateGameScreen extends StatefulWidget {
  const CreateGameScreen({super.key});

  @override
  State<CreateGameScreen> createState() => _CreateGameScreenState();
}

class _CreateGameScreenState extends State<CreateGameScreen> {
  static const _capacity = 4;

  final Future<List<Modalidade>> _modalidades = Api.modalidades();
  int _modalidadeId = 1;
  DateTime _date = DateUtils.dateOnly(DateTime.now()).add(const
Duration(days: 1));
```

```
  TimeOfDay _time = const TimeOfDay(hour: 10, minute: 0);
  String _level = 'Intermédio';
  String _format = 'Amigável';
  int _openSlots = 1;
  bool _aCriar = false;

  late final _dateCtrl = TextEditingController(text: dataCampo(_date));
  late final _timeCtrl = TextEditingController(text: _formatTime(_time));
  final _localCtrl = TextEditingController(text: 'Padel Club X');
  final _precoCtrl = TextEditingController(text: '8 €');

  @override
  void dispose() {
    _dateCtrl.dispose();
    _timeCtrl.dispose();
    _localCtrl.dispose();
    _precoCtrl.dispose();
    super.dispose();
  }

  String _formatTime(TimeOfDay t) => '${t.hour.toString().padLeft(2,
'0')}:${t.minute.toString().padLeft(2, '0')}';

  Future<void> _pickDate() async {
    final picked = await showDatePicker(
      context: context,
      initialDate: _date,
      firstDate: DateUtils.dateOnly(DateTime.now()),
      lastDate: DateTime(2100),
    );
    if (picked == null) return;
    setState(() {
      _date = picked;
      _dateCtrl.text = dataCampo(picked);
    });
  }

  Future<void> _pickTime() async {
    final picked = await showTimePicker(context: context, initialTime:
_time);
    if (picked == null) return;
    setState(() {
      _time = picked;
      _timeCtrl.text = _formatTime(picked);
    });
  }
```

```
  Future<void> _criar() async {
    // "8 €" → 8.0 · "7,50" → 7.5
    final preco = double.tryParse(_precoCtrl.text.replaceAll(RegExp(r'[^0-
9,.]'), '').replaceAll(',', '.')) ?? 0;
    setState(() => _aCriar = true);
    try {
      final id = await Api.criarJogo(
        modalidadeId: _modalidadeId,
        dataHora: DateTime(_date.year, _date.month, _date.day, _time.hour,
_time.minute),
        local: _localCtrl.text.trim(),
        nivel: _level,
        formato: _format,
        capacidade: _capacity,
        vagasAbertas: _openSlots,
        preco: preco,
      );
      if (!mounted) return;
      Navigator.of(context).pushReplacement(MaterialPageRoute(builder: (_) =>
GameDetailScreen(jogoId: id)));
    } on ApiException catch (erro) {
      if (mounted) showInfo(context, erro.message);
    } finally {
      if (mounted) setState(() => _aCriar = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: FillScrollView(
          padding: EdgeInsets.fromLTRB(20, topInset(context, 48), 20, 28 +
bottomInset(context)),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Row(
                children: [
                  CircleIconButton(
                    semanticLabel: 'Voltar',
                    onPressed: () => Navigator.of(context).pop(),
                    child: const SvgIcon(AppIcon.back),
                  ),
```

```
                const SizedBox(width: 12),
                const ScreenTitle('Criar jogo'),
              ],
            ),
            const SizedBox(height: 16),
            FutureBuilder<List<Modalidade>>(
              future: _modalidades,
              builder: (context, snapshot) {
                final lista = snapshot.data ?? const [Modalidade(id: 1,
nome: 'Padel', descricao: '')];
                return AppDropdown<int>(
                  label: 'Desporto',
                  value: _modalidadeId,
                  items: {for (final m in lista) m.id: m.nome},
                  onChanged: (v) => setState(() => _modalidadeId = v ??
_modalidadeId),
                );
              },
            ),
            const SizedBox(height: 16),
            Row(
              children: [
                Expanded(
                  flex: 3,
                  child: AppTextField(
                    label: 'Data',
                    controller: _dateCtrl,
                    weight: FontWeight.w600,
                    readOnly: true,
                    onTap: _pickDate,
                  ),
                ),
                const SizedBox(width: 12),
                Expanded(
                  flex: 2,
                  child: AppTextField(
                    label: 'Hora',
                    controller: _timeCtrl,
                    weight: FontWeight.w600,
                    readOnly: true,
                    onTap: _pickTime,
                  ),
                ),
              ],
            ),
            const SizedBox(height: 16),
```

```
              AppTextField(label: 'Local', controller: _localCtrl, weight:
FontWeight.w600),
              const SizedBox(height: 16),
              _ChoiceGroup(
                label: 'Nível',
                options: _levels,
                selected: _level,
                onSelected: (v) => setState(() => _level = v),
              ),
              const SizedBox(height: 16),
              _ChoiceGroup(
                label: 'Formato',
                options: _formats,
                selected: _format,
                onSelected: (v) => setState(() => _format = v),
              ),
              const SizedBox(height: 16),
              Row(
                crossAxisAlignment: CrossAxisAlignment.end,
                children: [
                  Expanded(
                    flex: 3,
                    child: LabeledField(
                      label: 'Vagas abertas',
                      child: FieldBox(
                        horizontalPadding: 4,
                        child: Row(
                          mainAxisAlignment: MainAxisAlignment.spaceBetween,
                          children: [
                            _StepperButton(
                              label: '−',
                              semanticLabel: 'Menos uma vaga',
                              onTap: _openSlots > 1 ? () => setState(() =>
_openSlots--) : null,
                            ),
                            Text('$_openSlots de $_capacity', style:
AppText.body(16, weight: FontWeight.w700)),
                            _StepperButton(
                              label: '+',
                              semanticLabel: 'Mais uma vaga',
                              onTap: _openSlots < _capacity - 1 ? () =>
setState(() => _openSlots++) : null,
                            ),
                          ],
                        ),
                      ),
```

```
                    ),
                  ),
                  const SizedBox(width: 12),
                  Expanded(
                    flex: 2,
                    child: AppTextField(
                      label: 'Preço por pessoa',
                      controller: _precoCtrl,
                      weight: FontWeight.w600,
                      keyboardType: const
TextInputType.numberWithOptions(decimal: true),
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 16),
              const Spacer(),
              AppButton(label: _aCriar ? 'A criar…' : 'Criar jogo',
onPressed: _aCriar ? null : _criar),
            ],
          ),
        ),
      ),
    );
  }
}
```

12.16 Ecrã 09 — `game_detail_screen.dart`

O ecrã passa a receber o `jogoId` e carrega `GET /jogos/:id`. O botão muda conforme o caso:

Situação	Botão	Ao carregar
Já estou no jogo	Ver o meu QR Code	Abre o Check-in com o meu código
Há vagas	Entrar no jogo	`POST /jogos/:id/entrar` e abre o Check-in
Não há vagas	Jogo cheio (desativado)	—

Aparece também uma linha nova, "2 jogadores de fora da app", quando o organizador indicou pessoas que não usam o WhoPlay. Este bloco substitui tudo, do início do ficheiro até à classe `_StatCell`; as classes `_StatCell`, `_OpenSlotRow` e `_DashedCirclePainter` ficam iguais.

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../data/models.dart';
import '../services/api.dart';
import '../theme/app_colors.dart';
import '../theme/app_theme.dart';
import '../utils/formatos.dart';
import '../widgets/app_icons.dart';
import '../widgets/async_view.dart';
import '../widgets/buttons.dart';
import '../widgets/common.dart';
import 'checkin_screen.dart';

/// 09 Detalhe do jogo
class GameDetailScreen extends StatefulWidget {
  const GameDetailScreen({super.key, required this.jogoId});

  final int jogoId;

  @override
  State<GameDetailScreen> createState() => _GameDetailScreenState();
}

class _GameDetailScreenState extends State<GameDetailScreen> {
  late Future<Jogo> _jogo = Api.jogo(widget.jogoId);
  bool _aEntrar = false;

  void _recarregar() => setState(() => _jogo = Api.jogo(widget.jogoId));

  Future<void> _entrar(Jogo jogo) async {
    setState(() => _aEntrar = true);
    try {
      final codigo = await Api.entrarNoJogo(jogo.id);
      if (!mounted) return;
      await Navigator.of(context).push(
        MaterialPageRoute(builder: (_) => CheckInScreen(jogo: jogo, codigo:
codigo)),
      );
      _recarregar(); // ao voltar, a lista "Quem vai" já me inclui
    } on ApiException catch (erro) {
      if (mounted) showInfo(context, erro.message);
    } finally {
      if (mounted) setState(() => _aEntrar = false);
    }
```

```
  }

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.lightStatusIcons,
      child: Scaffold(
        body: AsyncView<Jogo>(
          future: _jogo,
          onRetry: _recarregar,
          builder: (context, jogo) => _GameDetailBody(
            jogo: jogo,
            aEntrar: _aEntrar,
            onEnter: () => _entrar(jogo),
          ),
        ),
      ),
    );
  }
}

class _GameDetailBody extends StatelessWidget {
  const _GameDetailBody({required this.jogo, required this.aEntrar, required
this.onEnter});

  final Jogo jogo;
  final bool aEntrar;
  final VoidCallback onEnter;

  @override
  Widget build(BuildContext context) {
    final distancia = jogo.distanciaKm;
    // Lugares ocupados por pessoas que vão ao jogo mas não usam a app.
    final externos = jogo.ocupados - jogo.participantes.length;
    final String botao;
    if (jogo.euParticipo) {
      botao = 'Ver o meu QR Code';
    } else if (jogo.vagas > 0) {
      botao = aEntrar ? 'A entrar…' : 'Entrar no jogo';
    } else {
      botao = 'Jogo cheio';
    }
    final podeCarregar = !aEntrar && (jogo.euParticipo || jogo.vagas > 0);

    return FillScrollView(
      child: Column(
```

```
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Container(
            color: AppColors.forest,
            padding: EdgeInsets.fromLTRB(20, topInset(context, 48), 20, 24),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                CircleIconButton(
                  semanticLabel: 'Voltar',
                  background: AppColors.forestLight,
                  bordered: false,
                  onPressed: () => Navigator.of(context).pop(),
                  child: const SvgIcon(AppIcon.back, color: AppColors.white),
                ),
                const SizedBox(height: 14),
                SectionLabel('${jogo.modalidade} · ${jogo.formato}', color:
AppColors.lime, letterSpacing: 1.2),
                const SizedBox(height: 14),
                Text(
                  dataLonga(jogo.dataHora),
                  style: AppText.display(
                    34,
                    weight: FontWeight.w800,
                    color: AppColors.white,
                    letterSpacing: -0.8,
                    height: 1.08,
                  ),
                ),
                const SizedBox(height: 14),
                Row(
                  children: [
                    const SvgIcon(AppIcon.pin, size: 18, color:
AppColors.white),
                    const SizedBox(width: 6),
                    Flexible(
                      child: Text(
                        distancia == null || distancia < 0.1
                            ? jogo.localNome
                            : '${jogo.localNome} · a ${km(distancia)} de ti',
                        style: AppText.body(15, color: AppColors.white),
                      ),
                    ),
                  ],
                ),
              ],
```

```
            ),
          ),
          Expanded(
            child: Padding(
              padding: EdgeInsets.fromLTRB(20, 20, 20, 28 +
bottomInset(context)),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  IntrinsicHeight(
                    child: Row(
                      crossAxisAlignment: CrossAxisAlignment.stretch,
                      children: [
                        Expanded(child: _StatCell('Nível', jogo.nivel)),
                        const SizedBox(width: 8),
                        Expanded(child: _StatCell('Jogadores',
'${jogo.ocupados} de ${jogo.capacidade}')),
                        const SizedBox(width: 8),
                        Expanded(child: _StatCell('Preço',
'${euros(jogo.preco)} / pessoa')),
                      ],
                    ),
                  ),
                  const SizedBox(height: 18),
                  Section(
                    title: 'Quem vai',
                    child: ListCard(
                      children: [
                        for (final p in jogo.participantes)
_ParticipantRow(p),
                        if (externos > 0) _ExternalRow(externos),
                        if (jogo.vagas > 0) _OpenSlotRow(jogo.vagas),
                      ],
                    ),
                  ),
                  const SizedBox(height: 18),
                  const Spacer(),
                  Text(
                    jogo.euParticipo ? 'Já estás neste jogo.' : 'O
organizador é avisado assim que entrares.',
                    textAlign: TextAlign.center,
                    style: AppText.body(14, color: AppColors.muted),
                  ),
                  const SizedBox(height: 10),
                  AppButton(label: botao, onPressed: podeCarregar ? onEnter :
null),
```

```
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

Substituam também a classe `_ParticipantRow` e acrescentem a `_ExternalRow` logo a seguir:

```
class _ParticipantRow extends StatelessWidget {
  const _ParticipantRow(this.participante);

  final Participante participante;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
      child: Row(
        children: [
          Avatar(participante.nome.substring(0, 1), size: 40, fontSize: 17),
          const SizedBox(width: 12),
          Expanded(child: Text(participante.nome, style: AppText.body(16,
weight: FontWeight.w700))),
          if (participante.organizador)
            const Tag('Organizador', fontSize: 13, padding:
EdgeInsets.symmetric(horizontal: 10, vertical: 5))
          else
            Text(participante.nivel ?? '', style: AppText.body(14, color:
AppColors.muted)),
        ],
      ),
    );
  }
}

class _ExternalRow extends StatelessWidget {
  const _ExternalRow(this.count);

  final int count;

  @override
```

```
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
      child: Row(
        children: [
          Avatar('+$count', size: 40, fontSize: 15, background:
AppColors.chip, foreground: AppColors.muted),
          const SizedBox(width: 12),
          Text(
            count == 1 ? '1 jogador de fora da app' : '$count jogadores de
fora da app',
            style: AppText.body(16, weight: FontWeight.w600, color:
AppColors.muted),
          ),
        ],
      ),
    );
  }
}
```

12.17 Ecrã 08 — `map_screen.dart` (4 alterações)

O desenho do mapa e os marcadores dos jogadores continuam ilustrativos. O cartão de baixo e o marcador do clube passam a mostrar o próximo jogo aberto (`GET /jogos`).

1. Imports: juntem estes 3 aos que já existem.

```
import '../data/models.dart';
import '../services/api.dart';
import '../utils/formatos.dart';
```

2. Início do `_MapScreenState`: substituam o `_openGame` e a primeira linha do `build` por isto. O que antes estava dentro do `build` passa para o `_buildMap`.

```
  bool _within5km = false;
  final Future<List<Jogo>> _jogos = Api.jogos();

  void _openGame(Jogo jogo) => Navigator.of(context).push(
        MaterialPageRoute(builder: (_) => GameDetailScreen(jogoId: jogo.id)),
      );

  @override
  Widget build(BuildContext context) {
    // O próximo jogo aberto vem da API; os marcadores de jogadores ainda são
ilustrativos.
    return FutureBuilder<List<Jogo>>(
```

```
      future: _jogos,
      builder: (context, snapshot) => _buildMap(context, snapshot),
    );
  }

  Widget _buildMap(BuildContext context, AsyncSnapshot<List<Jogo>> snapshot)
{
    final jogos = snapshot.data ?? const <Jogo>[];
    final jogo = jogos.isEmpty ? null : jogos.first;
    return ColoredBox(
      // … o resto fica igual ao build antigo …
```

3. Marcador do clube: troquem o `Positioned(left: 70, top: 404, ...)` por:

```
                    if (jogo != null)
                      Positioned(
                        left: 70,
                        top: 404,
                        child: GestureDetector(
                          onTap: () => _openGame(jogo),
                          child: _VenueMarker(
                            '${jogo.localNome} · ${jogo.vagas} ${jogo.vagas
== 1 ? 'vaga' : 'vagas'}',
                          ),
                        ),
                      ),
```

4. Cartão de baixo: o `child:` do `SurfaceCard` passa a ser:

```
              child: jogo == null
                  ? Text(
                      snapshot.hasError
                          ? snapshot.error.toString()
                          : snapshot.connectionState == ConnectionState.done
                              ? 'Ainda não há jogos abertos perto de ti.'
                              : 'A procurar jogos…',
                      style: AppText.body(15, color: AppColors.muted),
                    )
                  : Column(
                      crossAxisAlignment: CrossAxisAlignment.stretch,
                      mainAxisSize: MainAxisSize.min,
                      children: [
                        Row(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            Expanded(
```

```
                              child: Column(
                                crossAxisAlignment: CrossAxisAlignment.start,
                                children: [
                                  Text('Jogo de ${jogo.modalidade}', style:
AppText.display(20)),
                                  const SizedBox(height: 3),
                                  Text(
                                    '${dataCurta(jogo.dataHora)} ·
${jogo.localNome}',
                                    style: AppText.body(14, color:
AppColors.muted),
                                  ),
                                  const SizedBox(height: 3),
                                  Text(
                                    jogo.distanciaKm == null
                                        ? jogo.nivel
                                        : '${jogo.nivel} · a
${km(jogo.distanciaKm!)} de ti',
                                    style: AppText.body(14, color:
AppColors.muted),
                                  ),
                                ],
                              ),
                            ),
                            const SizedBox(width: 12),
                            Tag('${jogo.ocupados} / ${jogo.capacidade}',
background: AppColors.lime),
                          ],
                        ),
                        const SizedBox(height: 12),
                        AppButton(
                          label: 'Ver jogo',
                          height: 48,
                          radius: 12,
                          fontSize: 16,
                          onPressed: () => _openGame(jogo),
                        ),
                      ],
                    ),
```

12.18 Ecrã 10 — `checkin_screen.dart` (5 alterações)

O QR Code passa a ser o `codigo_checkin` verdadeiro do jogador nesse jogo.

1. Imports: juntem `import '../data/models.dart';` e `import '../utils/formatos.dart';`.

2. O construtor e o campo passam a:

```
  const CheckInScreen({super.key, required this.jogo, required this.codigo});

  final Jogo jogo;

  /// Código único deste jogador neste jogo (coluna codigo_checkin).
  final String codigo;
```

3. `const SectionLabel('Padel · Amigável', ...)` → `SectionLabel('${jogo.modalidade} · ${jogo.formato}', color: AppColors.lime, letterSpacing: 1.2)` (sem `const`).
4. `'Sáb, 3 de outubro · 10:00'` → `dataMedia(jogo.dataHora)`, e `Text('Padel Club X', ...)` → `Text(jogo.localNome, ...)`.
5. No `QrImageView`: `data: checkInCode` → `data: codigo`.

12.19 `home_shell.dart`

Como o `IndexedStack` mantém os separadores abertos, os dados podiam ficar desatualizados (por exemplo, um convite enviado não aparecia nos Convites). A solução é um contador, `_refresh`, que muda sempre que se escolhe um separador. Mapa, Convites e Perfil usam-no na `key` e, quando a `key` muda, o Flutter cria o ecrã de novo, que volta a pedir os dados. Os Jogadores não usam o contador, para manterem os filtros.

Substituam a classe `_HomeShellState` por esta:

```
class _HomeShellState extends State<HomeShell> {
  late int _tab = widget.initialTab;

  // Muda sempre que se escolhe um separador: Mapa, Convites e Perfil voltam
a
  // pedir os dados à API (os Jogadores mantêm os filtros escolhidos).
  int _refresh = 0;

  static const _stackIndex = {
    HomeShell.playersTab: 0,
    HomeShell.mapTab: 1,
    HomeShell.invitesTab: 2,
    HomeShell.profileTab: 3,
  };

  void _onNavTap(int index) {
    if (index == AppBottomNav.createIndex) {
      Navigator.of(context).push(MaterialPageRoute(builder: (_) => const
CreateGameScreen()));
```

```
      return;
    }
    setState(() {
      _tab = index;
      _refresh++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return AnnotatedRegion(
      value: AppTheme.darkStatusIcons,
      child: Scaffold(
        body: IndexedStack(
          index: _stackIndex[_tab]!,
          children: [
            FindPlayersScreen(onInviteSent: () =>
_onNavTap(HomeShell.invitesTab)),
            MapScreen(key: ValueKey('mapa$_refresh')),
            InvitesScreen(key: ValueKey('convites$_refresh')),
            ProfileScreen(key: ValueKey('perfil$_refresh')),
          ],
        ),
        bottomNavigationBar: AppBottomNav(currentIndex: _tab, onTap:
_onNavTap),
      ),
    );
  }
}
```

12.20 Atualizar o teste e verificar

Como o `mock_data.dart` desapareceu, o teste passa a verificar os modelos e os formatos. Substituam `test/widget_test.dart` por:

```
import 'package:flutter_test/flutter_test.dart';
import 'package:whoplay/data/models.dart';
import 'package:whoplay/utils/formatos.dart';

void main() {
  test('Lê um jogador tal como a API o devolve', () {
    final jogador = Jogador.fromJson({
      'id': 2,
      'nome': 'Tchali',
      'modalidade': 'Padel',
      'nivel': 'Intermédio',
```

```
      'distancia_km': 1.8,
      'compatibilidade': 93,
      'criterios': [
        {'nome': 'Mesmo desporto', 'pontos': 30, 'maximo': 30},
        {'nome': 'Nível', 'pontos': 25, 'maximo': 25},
        {'nome': 'Disponibilidade', 'pontos': 16, 'maximo': 20},
        {'nome': 'Distância', 'pontos': 12, 'maximo': 15},
        {'nome': 'Objetivo', 'pontos': 10, 'maximo': 10},
      ],
      'avaliacao': 4.7,
      'objetivo': 'Jogos amigáveis',
      'disponivel_hoje': '19:00 – 21:00',
    });

    final soma = jogador.criterios.fold<int>(0, (total, c) => total +
c.pontos);
    expect(soma, jogador.compatibilidade);
    expect(km(jogador.distanciaKm), '1,8 km');
  });

  test('Formata datas como no mockup', () {
    final sabado = DateTime(2026, 10, 3, 10);
    expect(dataLonga(sabado), 'Sábado, 3 de outubro às 10:00');
    expect(dataCurta(sabado), 'Sáb, 3 Out · 10:00');
    expect(paraApi(sabado), '2026-10-03 10:00:00');
    expect(euros(8), '8 €');
    expect(horaCurta('13:30'), '13h30');
  });
}
```

E corram:

```
flutter analyze
flutter test
```

O resultado esperado é *No issues found!* e *All tests passed!*.

12.21 Correr tudo junto

São precisas 3 coisas a correr ao mesmo tempo:

1. **MySQL:** XAMPP → Start na linha MySQL.
2. **API:** num terminal, `cd backend` e depois `npm run dev`.
3. **App:** noutro terminal, na raiz, `flutter run -d chrome`.

Percurso de teste. Os passos 1, 2, 3, 5, 6 e 7 foram verificados no Chrome com estes resultados; os passos 4, 8 e 9 foram testados só na API (Parte 11):

- [ ] Login com sebastiao@email.com / 12345678 abre os Jogadores: Tchali 93%, Helio 85%, Cristiano 63%
- [ ] Ver perfil do Tchali mostra 30/30, 25/25, 16/20, 12/15 e 10/10
- [ ] Enviar convite leva aos Convites, com o Tchali em "Enviados"
- [ ] Aceitar o convite do Cristiano faz-o desaparecer de "Recebidos"
- [ ] Mapa → Ver jogo mostra o jogo de sábado com 3 de 4 e 1 vaga livre
- [ ] Ver o meu QR Code mostra um QR com o código da base de dados
- [ ] Botão + → Criar jogo abre o detalhe do jogo novo
- [ ] Perfil → Terminar sessão volta ao Login
- [ ] Criar conta → escolher desportos → Continuar abre o Perfil do novo utilizador

Trabalho em equipa e problemas comuns

Git: o que vai e o que não vai para o GitHub

Vai para o GitHub	Não vai (fica só no vosso PC)
`database/*.sql`	`backend/.env` (tem o `JWT_SECRET`)
`backend/src/`, `package.json`, `package-lock.json`, `.env.example`, `.gitignore`	`backend/node_modules/` (o `npm install` volta a criá-la)
`lib/`, `test/`, `pubspec.yaml`, `pubspec.lock`	`build/`, `.dart_tool/` (já estão no `.gitignore` do Flutter)

Primeira vez de cada pessoa depois do `git pull`:

```
cd backend
npm install
cp .env.example .env
```

Depois, corram o `schema.sql` e o `seed.sql` (Parte 4). Cada pessoa tem a sua própria base de dados no XAMPP: os dados de teste de um não aparecem no PC dos outros. O que se partilha pelo Git são os **scripts**.

Mudar a base de dados sem estragar o trabalho dos outros

1. Façam a mudança no `schema.sql` (por exemplo, uma coluna nova) e, se for preciso, no `seed.sql`.

2. Ajustem as rotas da API que usam essa tabela.
3. Façam commit dos 3 juntos, com uma mensagem clara, por exemplo `BD: coluna telefone em utilizadores`.
4. Avisem a equipa: depois do `git pull`, todos têm de voltar a correr os 2 scripts.

Numa app já publicada nunca se apaga a base de dados: escrevem-se *migrações* (`ALTER TABLE ...`). Para o projeto escolar, recriar tudo com os 2 scripts é mais simples e chega.

Divisão de trabalho sugerida (4 pessoas)

Guia	Responsável sugerido	Funcionalidade (da tabela ao ecrã)
WhoPlay — Guia 1: Contas e perfil	Sebastião	Contas e perfil + base comum (começa primeiro)
WhoPlay — Guia 2: Jogadores e compatibilidade	Tchali	Jogadores e compatibilidade
WhoPlay — Guia 3: Jogos e check-in	Hélio	Jogos e check-in
WhoPlay — Guia 4: Convites, avisos e integração	Cristiano	Convites, avisos e integração final

Em vez de cada pessoa ficar com uma camada (só a base de dados, só a API…), cada uma fica com uma funcionalidade completa: as suas tabelas, as suas rotas e os seus ecrãs. Assim cada pessoa testa o seu trabalho sozinha, sem esperar pelas outras. Cada guia diz que partes deste documento copiar, por que ordem, o que combinar com os colegas e como provar que está pronto.

Problemas comuns

Sintoma	Causa	Solução
A app mostra "Não foi possível ligar ao servidor. A API está a correr?"	A API está parada, ou o endereço está errado	`npm run dev` na pasta `backend`; no emulador Android é `10.0.2.2`, num telemóvel real é o IP do PC (Parte 12.3)
`/saude` responde "Erro interno do servidor" e o terminal mostra `ECONNREFUSED`	O MySQL do XAMPP está parado	XAMPP → Start na linha MySQL

Sintoma	Causa	Solução
O terminal mostra `ER_BAD_DB_ERROR: Unknown database 'whoplay'`	Ainda não correram o `schema.sql`	Parte 3
O terminal mostra `ER_ACCESS_DENIED_ERROR`	O utilizador ou a palavra-passe do `.env` estão errados	No XAMPP: `DB_USER=root` e `DB_PASSWORD=` vazio
`Error: Cannot find module 'express'`	Falta o `npm install`	`cd backend` e `npm install`
`EADDRINUSE: address already in use :::3000`	Já há uma API a correr noutro terminal	Fechem a outra (Ctrl+C) ou mudem o `PORT` no `.env`
401 "Sessão inválida ou expirada"	O token tem mais de 7 dias, ou o `JWT_SECRET` mudou	Façam login outra vez
400 "Define primeiro a tua localização no perfil."	O utilizador não tem latitude nem longitude	`PUT /perfil` com `latitude` e `longitude` (o Onboarding já o faz)
Acentos estragados ("Interm�dio")	Pedido enviado sem UTF-8 (curl no Windows) ou script SQL corrido sem `--default-character-set=utf8mb4`	Usem o Thunder Client ou a app; voltem a correr os scripts com a opção certa
A lista não mostra o que acabaram de mudar	O separador ainda tem os dados antigos	Puxem o ecrã para baixo para recarregar, ou mudem de separador e voltem
Erro de CORS na consola do Chrome	Falta o `app.use(cors())` no `server.js`	Confirmem a Parte 6.8

Próximos passos

- [ ] Ecrã "Editar perfil" que use `PUT /perfil`, `PUT /perfil/modalidades` e `PUT /perfil/disponibilidade`
- [ ] Passos 2 e 3 do Onboarding (nível e disponibilidade)
- [ ] Localização real com o pacote `geolocator`
- [ ] Guardar o token entre aberturas da app com o pacote `shared_preferences`
- [ ] Mapa real com o pacote `flutter_map` e os jogos de `GET /jogos`

- [ ] Leitor de QR para o organizador confirmar presenças (`POST /jogos/checkin`)